Sequential Randomization Tests Using e-values: Applications for trial monitoring

Fernando G. Zampieri[1]

[1]Department of Critical Care Medicine, University of Alberta, Edmonton, Canada

April 28, 2026

Abstract

Sequential monitoring of randomized trials traditionally relies on parametric assumptions or asymptotic approximations. We discuss a family of nonparametric sequential tests—collectively called e-RT—for binary, event-only, continuous, time-to-event, and multi-state endpoints. All variants derive validity solely from the randomization mechanism. Using a betting framework, each test constructs a test martingale by sequentially wagering on treatment assignments given observed outcomes. Under the null hypothesis of no treatment effect, the expected wealth cannot grow, guaranteeing anytime-valid Type I error control regardless of stopping rule. We prove validity for each variant, present simulation studies demonstrating calibration and power, and discuss the principled asymmetry in betting strategies across outcome types. We also introduce e-RTu, a universal abstraction that unifies all variants into a single domain-agnostic engine operating on binary signals. These methods provide a conservative, assumption-free complement to model-based sequential analyses.

Keywords: e-values, e-process, randomization test, sequential analysis, clinical trials

1 Introduction

Sequential monitoring of randomized controlled trials requires methods that control Type I errors regardless of when or why the monitoring stops. Traditional group-sequential designs rely on parametric assumptions and predetermined stopping boundaries. When these assumptions fail, or when trials adapt in ways not fully prespecified, validity guarantees may erode.

Monitoring clinical trials, especially in the context of acutely ill patients, is of paramount importance. Traditional methods include $\alpha - spending$ functions. When interim analyses are spaced, evidence grows unnoticed between the interim analyses. This may both delay the implementation of potentially useful strategies or prolong trials when safety signals arise early.

E-values and e-processes offer an alternative framework (Shafer, 2021; Vovk and Wang, 2021; Ramdas and Wang, 2025). An e-value is a measure of evidence against a null hypothesis with a specific property: its expected value under the null is at most 1. This simple constraint yields

anytime-valid inference: the Type I error guarantee holds at any stopping time, regardless of the stopping rule.

Duan et al. (2022) introduced interactive rank testing by betting (i-bet), which tests treatment effects by wagering on treatment assignments given observed outcomes. The intuition is discussed by Ramdas (2021). Under the null hypothesis, randomization ensures that assignments are independent of outcomes, so no betting strategy can systematically accumulate wealth.

Sokolova and Sokolov (2026) recently developed a practitioner-oriented framework for e-value monitoring in adaptive clinical trials, including design-calibrated binary e-processes, safe logrank monitoring, futility tools, platform-trial extensions, and an open-source implementation in the `evalinger` package. Their work emphasizes a design-calibrated view of e-process construction: the betting strategy is chosen to optimize expected evidence growth under a prespecified clinically meaningful alternative. They refer to this as a growth-rate-optimal (GROW) wager.

We propose e-RT (e-value Randomized Trial), a complementary family of methods for prospective sequential monitoring of randomized trials. Like i-bet (Duan et al., 2022), e-RT uses betting martingales for inference, but differs in key respects: e-RT monitors sequentially as patients enroll rather than analyzing completed trial data; it requires no covariates or working models; and its default wager policies can be learned from accumulating data rather than fixed by a hypothesized effect size. This yields methods with minimal assumptions suitable for real-time trial monitoring.

We present six variants: the binary e-RTb for event/no-event outcomes; e-RTe for event-only monitoring (requiring no non-event tracking); e-RTc for continuous endpoints; e-RTs for time-to-event data; e-RTms for multi-state trajectory data; and e-RTu, a universal abstraction under development. All share the same validity proof—the expected wealth multiplier is exactly 1 under the null—but differ in how they translate outcome data into wagers. We also distinguish the randomization-based validity engine from the wager policy. Wagers may be learned adaptively from accumulating data, preserving effect-size agnosticism, or prespecified from design alternatives to improve efficiency when those assumptions are credible. This distinction is especially relevant in sparse-update settings, where fixed or design-calibrated wagers may improve power without the same risk of catastrophic over-betting seen with dense every-patient updates.

2 Overall construction: the binary e-RT (e-RTb)

We begin with the binary case, which serves as the prototype for all subsequent variants. We introduce the method as "e-RT" in this section; henceforth it is referred to as **e-RTb** (binary) to distinguish it from the other members of the family.

2.1 Setup

Consider a sequential randomized trial with 1:1 allocation. At each enrollment $i = 1, 2, \ldots$, we observe:

- $T_i \in \{0, 1\}$: treatment assignment (0 = control, 1 = intervention)

- $Y_i \in \{0, 1\}$: binary outcome (0 = no event, 1 = event)

Treatment is assigned with known probability $p = P(T_i = 1)$, typically $p = 0.5$.

The null hypothesis is that treatment assignment has no effect on outcome:

$$H_0 : Y_i \perp T_i \text{ for all } i \tag{1}$$

Under this hypothesis, observing the outcome provides no information about which arm the patient was assigned to.

2.2 Wealth Process

Following Duan et al. (2022), we construct a wealth process by wagering on treatment assignments. After observing outcome Y_i but *before* learning treatment assignment T_i, we choose $\lambda_i \in [0, 1]$: the fraction wagered on intervention.

The wealth updates as:

$$W_i = W_{i-1} \times \begin{cases} \lambda_i / p & \text{if } T_i = 1 \\ (1 - \lambda_i)/(1 - p) & \text{if } T_i = 0 \end{cases} \tag{2}$$

starting from $W_0 = 1$. When we bet toward the correct arm, wealth grows; when wrong, it shrinks.

2.3 Betting Strategy

The validity guarantee holds for any betting strategy where λ_i depends only on $\mathcal{F}_{i-1}$. Power depends on choosing bets that grow wealth under the alternative. We use a strategy that learns the treatment effect from accumulating data.

Let:

$$\hat{\delta}_{i-1} = (\text{event rate in intervention}) - (\text{event rate in control}) \tag{3}$$

estimated from patients $1, \ldots, i-1$. The betting fraction is:

$$\lambda_i = \begin{cases} 0.5 + 0.5 \cdot c_i \cdot \hat{\delta}_{i-1} & \text{if } Y_i = 1 \\ 0.5 - 0.5 \cdot c_i \cdot \hat{\delta}_{i-1} & \text{if } Y_i = 0 \end{cases} \tag{4}$$

where $c_i \in [0, 1]$ ramps from 0 to 1 over a burn-in period:

$$c_i = \min\left(1, \max\left(0, \frac{i - n_0}{n_r}\right)\right) \tag{5}$$

with n_0 the burn-in period and n_r the ramp period. This prevents large bets when $\hat{\delta}_{i-1}$ is unstable due to small samples.

The logic: if $\hat{\delta} > 0$ (more events in intervention), then events suggest intervention and non-events

suggest control. If $\hat{\delta} < 0$ (fewer events in intervention), then events suggest control and non-events suggest intervention. The factor of 0.5 before $c_i \cdot \hat{\delta}_{i-1}$ ensures $\lambda_i \in [0, 1]$.

2.4 Worked Example

Consider a trial comparing intervention versus control, with a binary outcome (event or no event). Event is mortality, which is expected to be lower with intervention. Allocation is 1:1 ($p = 0.5$). Assume burn-in is complete ($c_i = 1$).

We look back at patients 1–199. Intervention arm has 100 patients, 35 events, so rate = 35.0%. Control arm has 99 patients, 40 events, so rate = 40.4%. $\hat{\delta}_{199} = 0.350 - 0.404 = -0.054$ (intervention looks protective).

Patient 200 has an event (dies). Where is this patient likely from? Events are more common in control (40.4% vs 35.0%), so probably control. We bet $\lambda = 0.5 + 0.5 \times (-0.054) = 0.473$ on intervention and $1 - \lambda = 0.527$ on control. Assignment revealed: control. We guessed right. Multiplier: $0.527/0.5 = 1.054$. Wealth grows 5.4%.

Patient 201: Update counts—intervention has 100 patients, 35 events (35.0%); control has 100 patients, 41 events (41.0%). $\hat{\delta}_{200} = -0.060$. Patient 201 has no event. Non-events are more common in intervention (65.0% vs 59.0%), so probably intervention. Bet: $\lambda = 0.5 - 0.5 \times (-0.060) = 0.530$. Assignment revealed: intervention. Multiplier: $0.530/0.5 = 1.060$. Wealth grows 6.0%.

Patient 202: Update counts—intervention has 101 patients, 35 events (34.7%); control has 100 patients, 41 events (41.0%). $\hat{\delta}_{201} = -0.063$. Patient 202 has an event. Bet toward control: $\lambda = 0.5 + 0.5 \times (-0.063) = 0.469$. Assignment revealed: intervention. Wrong guess. Multiplier: $0.469/0.5 = 0.938$. Wealth **shrinks** 6.2%.

Cumulative wealth:

$$W_{202} = W_{199} \times 1.054 \times 1.060 \times 0.938 = W_{199} \times 1.048 \tag{6}$$

Despite one wrong guess, wealth grew 4.8% over these three patients. Under the alternative, correct guesses outnumber incorrect ones on average and wealth grows. Under the null, right and wrong guesses balance out and wealth fluctuates around 1.

2.5 Validity

Theorem 1. *Under the null hypothesis, the wealth process (W_n) is a nonnegative martingale.*

Proof. Under the null, outcome and treatment are independent. After observing outcome Y_i, treatment assignment remains a coin flip with $P(T_i = 1) = p$. The expected multiplier given any bet λ_i is:

$$\mathbb{E}[\text{multiplier} \mid \lambda_i] = p \times \frac{\lambda_i}{p} + (1 - p) \times \frac{1 - \lambda_i}{1 - p} \tag{7}$$

$$= \lambda_i + (1 - \lambda_i) = 1 \tag{8}$$

Thus $\mathbb{E}[W_i \mid W_{i-1}, \lambda_i] = W_{i-1}$. □

Corollary 1. *By Ville's inequality (Ville, 1939), for any nonnegative martingale starting at 1:*

$$\Pr_{H_0}\left(\sup_{n\geq 1} W_n \geq \frac{1}{\alpha}\right) \leq \alpha \tag{9}$$

Thus rejecting when wealth ever exceeds $1/\alpha$ controls Type I error at level α, regardless of when or why monitoring stops.

3 Simulation Studies

We evaluated operating characteristics or e-RT by simulation. For each scenario, we calculated the sample size required for a chi-square test to achieve the target power at $\alpha = 0.05$, then ran 5,000 simulated trials at that sample size. We used burn-in = 50 patients and ramp = 100 patients. Control arm event rate was 40% in all scenarios.

3.1 Results

Table 1 presents Type I error and power for trials designed to detect 5% or 10% absolute risk reductions (ARR) with 80% or 90% power.

Table 1: Operating Characteristics for e-RTb (Binary)

ARR	Target Power	n	Type I Error	e-RTb Power	Median Crossing
5%	80%	2942	0.032	48.6%	1392 (47%)
10%	80%	712	0.021	50.4%	401 (56%)
5%	90%	3938	0.035	62.8%	1842 (47%)
10%	90%	954	0.025	65.9%	478 (50%)

Type I error was well controlled across all scenarios ($0.021 - 0.035$), below the nominal $\alpha = 0.05$. This confirms the theoretical guarantee from the martingale property.

Power was approximately 50% for trials designed with 80% power, and 63–66% for trials designed with 90% power. When the process rejected the null, it did so at approximately half the planned sample size (median crossing 47–56% of total enrollment).

3.2 Interpretation

This method seems to be appealing as a continuous monitoring tool. A trial designed for 80% power with a traditional test gains a 50% chance of stopping early, at roughly the halfway point. If the threshold is not crossed, the trial proceeds to completion and the planned analysis is conducted with no penalty.

This represents a free option: anytime-valid monitoring with no alpha spending and no pre-specified interim looks. The "cost" is that this method alone has lower power than a fixed-sample

test. But when layered on top of a properly powered trial, it provides early stopping when effects are larger than anticipated.

3.3 Trajectory Examples

Figure 1 shows example wealth trajectories from 30 simulated trials under the null hypothesis (both arms 40% event rate). Sample sizes correspond to trials designed for 80% power (n = 712, left) and 90% power (n = 954, right) to detect a 10% ARR. Under the null, wealth fluctuates randomly around 1. Some trajectories temporarily rise toward the threshold but eventually drift back down. The downward drift over time reflects the accumulating "cost" of betting on noise.

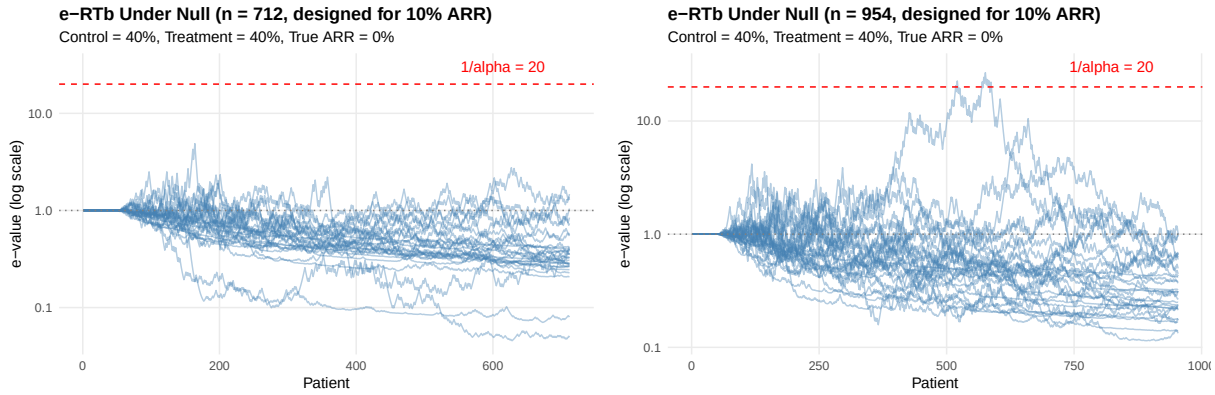

Figure 1: Wealth trajectories under the null hypothesis. Left: n = 712 (80% power design). Right: n = 954 (90% power design). Dashed red line: rejection threshold ($1/\alpha = 20$). Dotted gray line: neutral (wealth = 1). Under the null, no trajectory crosses the threshold.

Figure 2 shows trajectories under the alternative hypothesis (40% vs 30% event rates, true ARR = 10%). With a real treatment effect, wealth grows systematically. Most trajectories cross the rejection threshold before enrollment completes, and many reach values far exceeding 20 — providing strong evidence against the null.

4 Event-Only Monitoring (e-RTe)

4.1 Motivation

The e-RTb requires knowing both the treatment arm and the outcome for every enrolled patient. In practice, this means a coordinator must ascertain whether each patient experienced the event or not—requiring follow-up, data entry, and outcome adjudication for all patients, including the majority who do not experience the event.

In some settings, events of interest are reliably captured but non-events may not be systematically followed. For example, in studies of mortality in acutely ill patients embedded in electronic medical records (EMR), a death is an unmissable event; a survivor at day 28 requires active confirmation. Similarly, in oncology trials monitoring disease progression, or in cardiovascular trials

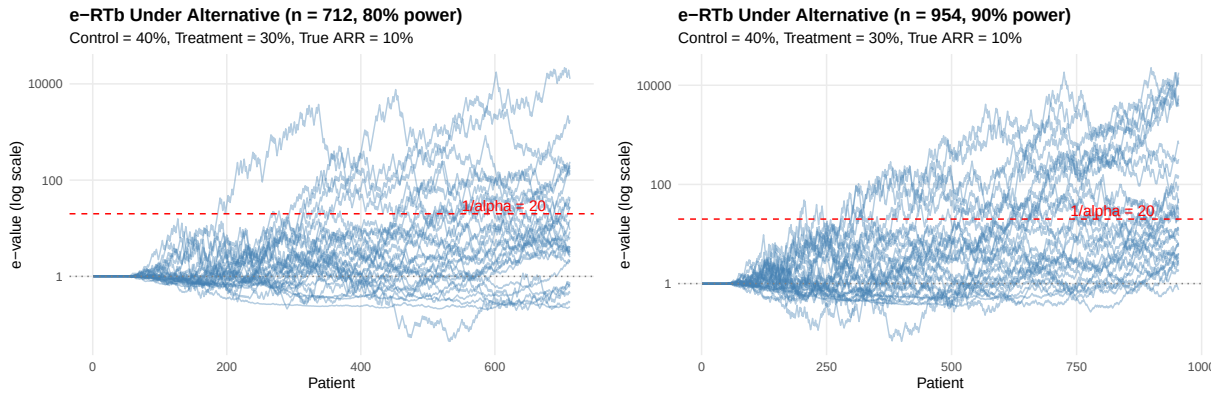

Figure 2: E-processes trajectories under the alternative hypothesis (true ARR = 10%). Left: n = 712 (80% power design). Right: n = 954 (90% power design). Dashed red line: rejection threshold ($1/\alpha = 20$). Under the alternative, approximately half of trajectories cross the threshold, typically around the midpoint of enrollment.

tracking myocardial infarction, the event is often captured with high fidelity while confirming event-free status requires active follow-up. This motivates a simpler variant: **e-RTe monitors only the stream of events, ignoring non-events entirely.**

The trade-off is explicit: by discarding information about non-events, we lose statistical efficiency (approximately 2.5× sample size inflation compared with frequentist methods). What we gain is operational simplicity—the only data required are the arm labels on events, in the order they occur. No denominators, no follow-up windows, no non-event tracking.

4.2 Null Hypothesis

Consider a trial with 1:1 randomization. Under the null hypothesis of no treatment effect on the event rate, both arms have the same event rate. If both arms have equal event rates and equal enrollment, then each event is equally likely to come from either arm:

$$H_0 : P(\text{event from treatment} \mid \text{an event occurred}) = 0.5 \tag{10}$$

In plain language: if the treatment does not affect the event rate, then looking at the arm label on an event is like flipping a fair coin. Treatment events and control events should arrive in roughly equal numbers.

This reduces trial monitoring to a sequential test of a Bernoulli coin: is the fraction of treatment events equal to 0.5? We call this the "event coin."

4.3 Algorithm

The algorithm maintains two counters: d_{trt} and d_{ctrl}, both initialized to zero. For each event $i = 1, 2, \ldots$:

Step 1: Estimate the coin bias from past data. Using all events before the current one, compute the plug-in estimator:

$$\hat{p}_{i-1} = \frac{d_{\text{trt}}}{d_{\text{trt}} + d_{\text{ctrl}}} \tag{11}$$

If no events have been observed yet, set $\hat{p} = 0.5$. In words: $\hat{p}$ is the running fraction of events that came from the treatment arm. Under the null, this should hover around 0.5. Under the alternative (if treatment helps), this should drift below 0.5.

Step 2: Compute the wager. The betting fraction is:

$$\lambda_i = 0.5 + c_i \cdot (\hat{p}_{i-1} - 0.5) \tag{12}$$

clamped to $[0.001, 0.999]$, where c_i is the same ramp function as before:

$$c_i = \min\left(1, \max\left(0, \frac{i - n_0}{n_r}\right)\right) \tag{13}$$

with burn-in $n_0 = 30$ events and ramp $n_r = 50$ events. During the burn-in, $c_i = 0$ and $\lambda_i = 0.5$ (a neutral bet—no information yet). After the ramp completes, $c_i = 1$ and $\lambda_i = \hat{p}_{i-1}$: the wager fully reflects the observed event proportion.

In plain language: we bet proportionally to what we have learned so far. If 40% of past events were from treatment ($\hat{p} = 0.40$), we set $\lambda = 0.40$. This means we place more of our wager on the "control event" side, because past data suggest control events are more common.

At full ramp ($c_i = 1$), this is the *full Kelly bet*—the wager that maximizes the expected logarithmic growth rate of wealth. This contrasts with the half-Kelly strategy used in e-RTb. Full Kelly is viable here because events are sparse: a trial with 2,000 patients and 15% event rate produces approximately 300 events. Over-betting compounds across only a few hundred multiplications, causing gradual wealth erosion rather than the catastrophic collapse seen with dense (every-patient) updates.

Step 3: Update wealth.

$$W_i = W_{i-1} \times \begin{cases} \lambda_i / 0.5 & \text{if the event is from the treatment arm} \\ (1 - \lambda_i)/0.5 & \text{if the event is from the control arm} \end{cases} \tag{14}$$

starting from $W_0 = 1$.

In plain language: we place λ_i on "treatment event" and $1 - \lambda_i$ on "control event." The null hypothesis pays $2:1$ on the correct side (a fair payout for a 0.5-probability event). If we guessed correctly, wealth grows; if we guessed wrong, wealth shrinks.

Step 4: Update counters after betting. Increment d_{trt} or d_{ctrl} depending on which arm the event came from. This ordering—bet first, then update—ensures that the estimate $\hat{p}_{i-1}$ uses only past information, maintaining the martingale property.

4.4 Worked Example: A New Event Arrived, Let Us Update Together

Consider a trial comparing a new treatment against standard of care in the ICU, where the event of interest is mortality. Suppose 80 deaths have been observed so far. Burn-in (30) and ramp (50) are complete, so $c_i = 1$. Current counts: $d_{\text{trt}} = 33$, $d_{\text{ctrl}} = 47$.

Event 81 arrives. Let us walk through the update.

Step 1: $\hat{p}_{80} = 33/(33 + 47) = 33/80 = 0.4125$. Interpretation: so far, 41.25% of events came from the treatment arm. This is below 50%, suggesting treatment may be protective.

Step 2: $\lambda_{81} = 0.5 + 1.0 \times (0.4125 - 0.5) = 0.4125$. We place 41.25% of our bet on "treatment event" and 58.75% on "control event." We are leaning toward control because past evidence suggests treatment events are less frequent.

Step 3: The arm is revealed: it is a **control** event.

$$\text{Multiplier} = \frac{1 - 0.4125}{0.5} = \frac{0.5875}{0.5} = 1.175 \tag{15}$$

Wealth grows by 17.5%. Our bet was correct—we leaned toward control, and indeed it was a control event.

Step 4: Update $d_{\text{ctrl}} = 48$. Now $\hat{p}_{81} = 33/81 = 0.407$.

Had the event been from the treatment arm instead:

$$\text{Multiplier} = \frac{0.4125}{0.5} = 0.825 \tag{16}$$

Wealth would have shrunk by 17.5%. Our lean toward control would have been wrong.

Under the null, treatment and control events arrive with equal probability, so wins and losses balance on average. Under the alternative ($\hat{p} < 0.5$), control events are genuinely more frequent, and wealth grows systematically.

4.5 Validity

Theorem 2. *Under the null hypothesis $P(\text{event from treatment} \mid \text{event}) = 0.5$, the wealth process (W_i) is a nonnegative martingale.*

Proof. Condition on all past information and the current wager λ_i. Under the null, the probability that event i is from the treatment arm is exactly 0.5, independently of all past data. The expected wealth multiplier is:

$$\mathbb{E}\left[\frac{W_i}{W_{i-1}} \,\middle|\, \lambda_i\right] = 0.5 \times \frac{\lambda_i}{0.5} + 0.5 \times \frac{1 - \lambda_i}{0.5} \tag{17}$$

$$= \lambda_i + (1 - \lambda_i) = 1 \tag{18}$$

This is the same identity as in the binary case. Since the expected multiplier is exactly 1, wealth cannot systematically grow under the null, regardless of how λ_i was chosen (as long as it depends only on past events). □

By Ville's inequality, $\Pr_{H_0}(\sup_{i\geq 1} W_i \geq 1/\alpha) \leq \alpha$. Rejecting when wealth crosses $1/\alpha$ controls Type I error at any stopping time.

4.6 Bidirectionality

The method automatically detects both benefit and harm without pre-specifying a direction:

- If $\hat{p} < 0.5$ (fewer treatment events than expected): the method bets on control events being more common, and wealth grows when treatment is indeed protective.
- If $\hat{p} > 0.5$ (more treatment events than expected): the method bets on treatment events being more common, and wealth grows when treatment is harmful.

No investigator input about the expected direction is needed. The adaptive wager discovers the direction from the data.

4.7 Signal Concentration

A key property of e-RTe is that it can outperform the full-sample e-RTb when the baseline event rate is low. The intuition is that events *concentrate* the treatment signal.

Consider a trial where the control event rate is 25% and the treatment event rate is 20%, yielding a 5 percentage-point absolute risk reduction (ARR). In e-RTb, the signal per patient is diluted: most patients do not experience the event, and only the 20–25% who do carry information about differential event rates. The observed risk difference across all patients is 5 percentage points.

In e-RTe, only events are observed. The event-coin probability is:

$$p_{\text{alt}} = \frac{p_{\text{trt}}}{p_{\text{trt}} + p_{\text{ctrl}}} = \frac{0.20}{0.20 + 0.25} = 0.444 \tag{19}$$

This is an 11.2-point tilt from 0.5—more than double the 5-point ARR. The signal is concentrated because events filter out the uninformative non-events.

This advantage diminishes as the baseline event rate increases. At higher event rates, e-RTb sees more informative events per patient, and the event-coin tilt shrinks because both numerator and denominator grow:

Table 2: Signal Concentration: Event-Coin Tilt vs. ARR for a 5pp Risk Reduction

Baseline Event Rate	Treatment Event Rate	Event Coin p_{alt}	Tilt from 0.5	Tilt / ARR
10%	5%	0.333	16.7 pp	3.33×
15%	10%	0.400	10.0 pp	2.00×
20%	15%	0.429	7.1 pp	1.43×
25%	20%	0.444	5.6 pp	1.11×
30%	25%	0.455	4.5 pp	0.91×
35%	30%	0.462	3.8 pp	0.77×
40%	35%	0.467	3.3 pp	0.67×

The crossover with e-RTb occurs at approximately 25% baseline event rate (Table 4). Below that, the concentrated event-coin signal more than compensates for the smaller number of observations; above it, e-RTb's access to all patients provides the advantage.

4.8 Simulation Studies

We evaluated e-RTe operating characteristics using the same simulation framework as for e-RTb. For each scenario, we calculated the frequentist sample size (two-proportion z-test) and applied a 2.5× inflation factor to account for the reduced efficiency of event-only monitoring. We used burn-in = 30 events, ramp = 50 events, threshold $1/\alpha = 20$, and 2,000 simulations per scenario. The simulations use mortality as the event of interest, but the method applies to any binary event.

Table 3: Operating Characteristics for e-RTe (Event-Only)

Baseline	ARR	Event Coin	N (freq)	N (e-RTe)	Type I	Power	Median Crossing
15%	5pp	0.400	1,372	3,430	2.20%	90.3%	179 events (42%)
15%	10pp	0.250	282	705	0.35%	59.6%	63 events (89%)
25%	5pp	0.444	2,188	5,470	2.90%	86.0%	490 events (40%)
25%	10pp	0.375	500	1,250	2.15%	87.1%	123 events (49%)
35%	5pp	0.462	2,754	6,885	3.05%	76.1%	1,036 events (46%)
35%	10pp	0.417	658	1,645	3.15%	81.1%	232 events (47%)

Type I error was well controlled across all scenarios (0.35–3.15%), consistently below the nominal $\alpha = 0.05$. Power ranged from 59.6% to 90.3%, with the strongest performance at low baseline event rates where the signal concentration effect is most pronounced: at 15% baseline event rate with a 5pp ARR, the event-coin tilt of 10 points from 0.5 yielded 90.3% power. At higher baseline rates (35%), the event-coin tilt shrinks and power decreases, consistent with the signal concentration analysis above.

When the threshold was crossed, it occurred at approximately 40–50% of total expected events for most scenarios. The exception is 15% baseline with 10pp ARR, where the very strong event-coin tilt (0.250, a 25-point deviation from 0.5) drives rapid crossing at the cost of requiring very few events overall (median 63), so the 89% figure reflects that the total expected event count (71) is small and crossing happens close to the end.

4.9 Trajectory Examples

Figure 3 shows example wealth trajectories for e-RTe. Under the null (left), the event coin is fair and wealth fluctuates around 1. Under the alternative (right), treatment events arrive less frequently than control events, the adaptive wager learns this imbalance, and wealth grows.

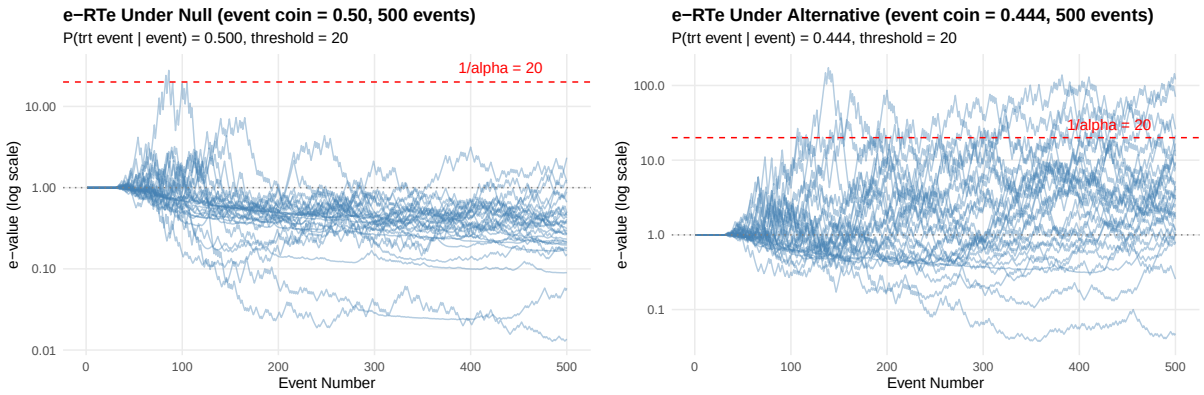

Figure 3: Trajectories of the e-RTe process (25% baseline event rate, 5pp ARR, 500 events). Left: under the null hypothesis (event coin = 0.50), wealth fluctuates randomly. Right: under the alternative hypothesis, wealth grows as the adaptive wager learns the event-coin imbalance. Dashed red line: rejection threshold ($1/\alpha = 20$).

4.10 Head-to-Head Comparison with e-RTb

To quantify the signal concentration crossover, we ran both e-RTe and e-RTb on the *same* simulated trials across a range of baseline event rates with a fixed 5pp ARR. For each baseline rate, we computed the frequentist sample size (two-proportion z-test, 80% power), enrolled that many patients, and analyzed the data with both methods: e-RTb processed all patients; e-RTe processed only the event stream. We used 2,000 simulations per scenario.

Table 4: Head-to-Head Comparison: e-RTe vs. e-RTb (5pp ARR, same trial data)

Baseline	Event Coin	N	Events	e-RTb Power	e-RTe Power	Δ	Winner
10%	0.333	870	66	12.2%	8.4%	−3.7pp	e-RTb
15%	0.400	1,372	172	26.7%	44.6%	+17.9pp	e-RTe
20%	0.429	1,812	318	32.4%	44.0%	+11.6pp	e-RTe
25%	0.444	2,188	493	40.6%	41.3%	+0.7pp	∼Tied
30%	0.455	2,502	689	42.3%	38.4%	−3.9pp	e-RTb
35%	0.462	2,754	896	45.3%	36.1%	−9.2pp	e-RTb
40%	0.467	2,942	1,104	48.8%	33.1%	−15.6pp	e-RTb

The crossover occurs at exactly 25% baseline event rate, confirming the analytical prediction from Table 2. At 15–20% baseline, e-RTe outperforms e-RTb by 12–18 percentage points despite seeing fewer observations, because the event-coin tilt (10–7 points from 0.5) more than compensates for the smaller sample. Above 30%, e-RTb's access to all patients provides an increasingly large advantage.

The 10% exception. At 10% baseline, the frequentist $N = 870$ produces only ∼66 expected events—fewer than the burn-in (30) plus ramp (50) = 80 events required for e-RTe to reach full betting strength. The e-RTe process literally never completes its learning phase, so it cannot leverage

the strong 16.7-point event-coin tilt. This highlights a practical constraint: e-RTe requires sufficient events (> 80) to be effective.

The 10pp ARR case. With a larger effect (10pp ARR), the frequentist N shrinks dramatically (282 at 15% baseline), producing even fewer events. e-RTb dominates across all tested baseline rates because the larger per-patient effect can be detected with fewer observations, while e-RTe cannot accumulate enough events during its burn-in/ramp phase.

Practical guidance. Use e-RTe when (i) the baseline event rate is 15–25%, (ii) the expected ARR is modest ($\leq$5pp), and (iii) ascertainment of non-events is impractical. Above 25% baseline or with larger expected effects, e-RTb is more powerful.

4.11 Adaptive and Design-Fixed Wager Policies

The preceding simulations used the default wager policies: adaptive half-Kelly for e-RTb and adaptive full-Kelly for e-RTe. In Version 8 we explicitly separate the validity engine from the wager policy. The martingale argument requires only that the wager be predictable: it may be learned from prior trial data, or it may be fixed in advance from the design alternative. This distinction parallels Sokolova and Sokolov (2026), where the e-process is calibrated to a clinically meaningful design effect.

In the terminology of Sokolova and Sokolov (2026), a GROW wager is the value $\lambda^\star$ that maximizes the expected log-growth of the e-process under a specified design alternative:

$$\lambda^\star = \arg\max_{\lambda} \; \mathbb{E}_{\text{design}}\{\log M(\lambda)\}, \tag{20}$$

where $M(\lambda)$ is the one-step e-process multiplier. In their binary paired-comparison construction, this wager is computed from the design alternative before monitoring and then held fixed during the e-value run. This is analogous to a frequentist design effect used for sample-size planning: it improves efficiency when the design alternative is close to the truth, but can lose power when the effect is misspecified. The design-fixed e-RT policies below adopt the same planning philosophy, while retaining the e-RT assignment-prediction construction rather than the paired-comparison construction.

For e-RTb, the design-fixed wager uses the posterior assignment probabilities implied by prespecified event rates. With 1:1 randomization and design rates p_T and p_C,

$$\lambda_{\text{event}} = \Pr(T = 1 \mid Y = 1) = \frac{p_T}{p_T + p_C}, \qquad \lambda_{\text{non-event}} = \Pr(T = 1 \mid Y = 0) = \frac{1 - p_T}{(1 - p_T) + (1 - p_C)}. \tag{21}$$

For e-RTe, the design-fixed wager is the corresponding event-coin probability, $\Pr(T = 1 \mid \text{event}) = p_T/(p_T + p_C)$. Thus e-RTb fixes both an event and non-event wager, whereas e-RTe fixes only the event wager.

We ran 1,000 simulations per scenario using a fixed seed. Sample sizes were anchored to the usual fixed-sample two-proportion calculation in R (`power.prop.test`, 80% power, $\alpha = 0.05$). Both e-RTb and e-RTe were evaluated at the same enrolled-patient N; no event-only inflation was used in this comparison. Under the null, we reused the design N for 5pp and 10pp ARR alternatives. Under the alternative, we compared adaptive wagering against fixed wagers that underestimated, matched, or overestimated the true ARR. An oracle full-Kelly row is included only as a simulation benchmark.

Table 5: Type I error for adaptive and design-fixed wager policies at the same enrolled-patient sample sizes. Each row summarizes 1,000 simulated null trials with $p_C = 0.40$ and $\alpha = 0.05$. Sample sizes were obtained from the usual fixed-sample two-proportion calculation with 80% power; no event-only inflation was used for e-RTe in this comparison.

Endpoint	Scenario	Policy	Wager ARR	N	Events	Type I
e-RTb	Null 5.0pp design	Adaptive	–	2,942	–	3.5%
e-RTb	Null 5.0pp design	Fixed 5pp	5.0pp	2,942	–	3.1%
e-RTb	Null 5.0pp design	Fixed 10pp	10.0pp	2,942	–	5.1%
e-RTb	Null 10.0pp design	Adaptive	–	712	–	1.8%
e-RTb	Null 10.0pp design	Fixed 5pp	5.0pp	712	–	0.4%
e-RTb	Null 10.0pp design	Fixed 10pp	10.0pp	712	–	2.8%
e-RTe	Null 5.0pp design	Adaptive	–	2,942	1,177	3.0%
e-RTe	Null 5.0pp design	Fixed 5pp	5.0pp	2,942	1,177	2.8%
e-RTe	Null 5.0pp design	Fixed 10pp	10.0pp	2,942	1,177	4.5%
e-RTe	Null 10.0pp design	Adaptive	–	712	285	2.1%
e-RTe	Null 10.0pp design	Fixed 5pp	5.0pp	712	285	0.0%
e-RTe	Null 10.0pp design	Fixed 10pp	10.0pp	712	285	2.6%

The key result is that design-fixed full-Kelly wagers can substantially increase power when the design effect is close to the truth, while retaining Type I error control in these simulations. However, misspecification matters. Underestimating the effect is usually conservative; overestimating the effect can produce earlier crossings among successful trials but lower overall power, especially for e-RTe at a true 5pp ARR. Because e-RTe observes only events, the same enrolled-patient N produces fewer betting opportunities than e-RTb; this explains why the same-N comparison is less favorable to e-RTe than the inflated event-only simulation above. Overall, design-fixed wagers appear useful as an optional mode rather than as a replacement for adaptive effect-size-agnostic monitoring.

5 Continuous Outcomes

The e-RTb treats each patient as a single Bernoulli trial: the outcome (event vs. no event) is observed, and we bet on which arm that patient came from. For continuous endpoints, the logic is the same but the signal is richer. Each patient now contributes a continuous measurement (for example, ventilator-free days, change in biomarker, or a physiologic score), and the betting strategy uses how extreme that value is relative to past data. Therefore, defining wager is slightly more

Table 6: Power for adaptive and design-fixed wager policies at the same enrolled-patient sample sizes. Each row summarizes 1,000 simulated trials with $p_C = 0.40$ and $\alpha = 0.05$. Fixed policies use full-Kelly wagers calibrated to the listed wager ARR; adaptive e-RTb uses half-Kelly and adaptive e-RTe uses full-Kelly.

Endpoint	Scenario	Policy	Wager ARR	N	Events	Power	Median crossing
e-RTb	True 5.0pp	Adaptive	–	2,942	–	49.9%	1,523
e-RTb	True 5.0pp	Fixed under	2.5pp	2,942	–	50.1%	2,084
e-RTb	True 5.0pp	Fixed matched	5.0pp	2,942	–	72.3%	1,335
e-RTb	True 5.0pp	Fixed over	10.0pp	2,942	–	56.7%	813
e-RTb	True 5.0pp	Oracle	5.0pp	2,942	–	74.6%	1,428
e-RTb	True 10.0pp	Adaptive	–	712	–	52.8%	402
e-RTb	True 10.0pp	Fixed under	5.0pp	712	–	41.8%	569
e-RTb	True 10.0pp	Fixed matched	10.0pp	712	–	69.1%	417
e-RTb	True 10.0pp	Fixed over	15.0pp	712	–	67.4%	325
e-RTb	True 10.0pp	Oracle	10.0pp	712	–	71.7%	404
e-RTe	True 5.0pp	Adaptive	–	2,942	1,104	31.0%	530
e-RTe	True 5.0pp	Fixed under	2.5pp	2,942	1,104	14.5%	948
e-RTe	True 5.0pp	Fixed matched	5.0pp	2,942	1,104	53.5%	668
e-RTe	True 5.0pp	Fixed over	10.0pp	2,942	1,104	46.8%	387
e-RTe	True 5.0pp	Oracle	5.0pp	2,942	1,104	50.6%	663
e-RTe	True 10.0pp	Adaptive	–	712	250	32.7%	165
e-RTe	True 10.0pp	Fixed under	5.0pp	712	250	5.7%	232
e-RTe	True 10.0pp	Fixed matched	10.0pp	712	250	43.7%	189
e-RTe	True 10.0pp	Fixed over	15.0pp	712	250	49.6%	148
e-RTe	True 10.0pp	Oracle	10.0pp	712	250	45.3%	182

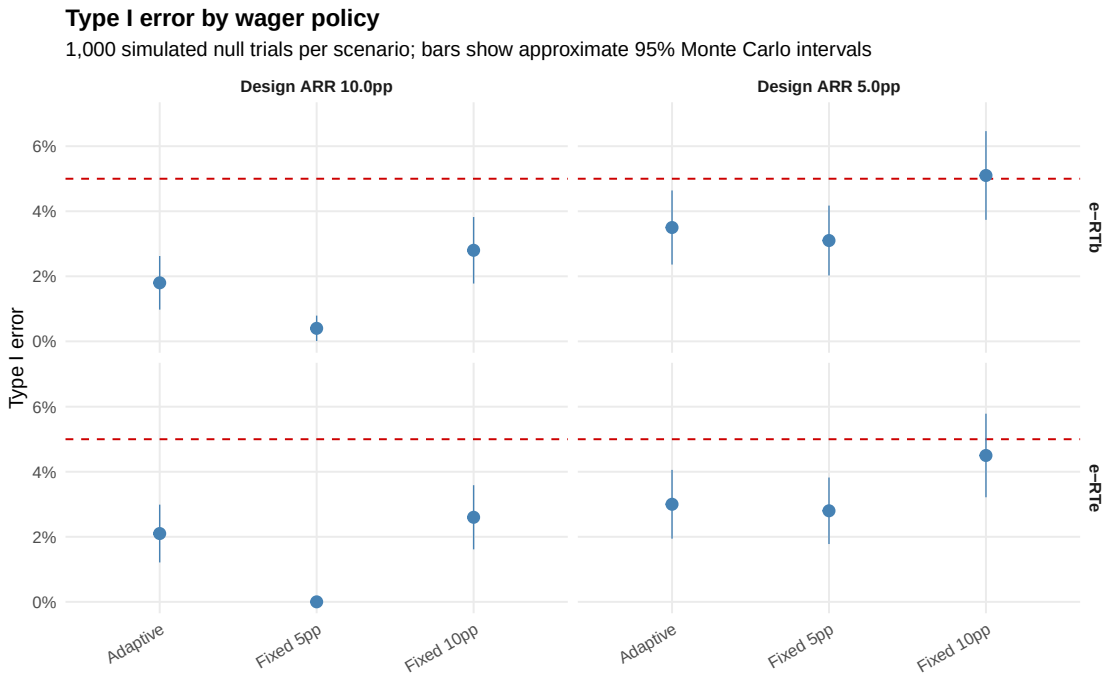

Figure 4: Type I error for adaptive and design-fixed wager policies. Each point summarizes 1,000 null simulations. The dashed red line is the nominal $\alpha = 0.05$ threshold.

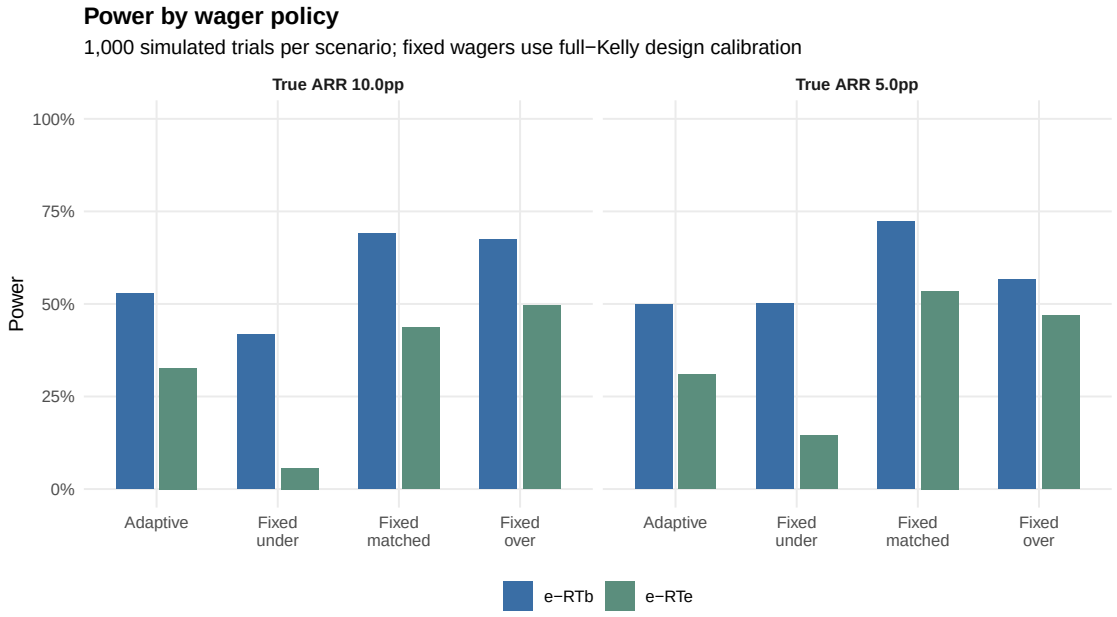

Figure 5: Power for adaptive and design-fixed wager policies. Fixed matched wagers are calibrated to the true ARR; underestimated and overestimated wagers deliberately misspecify the design effect.

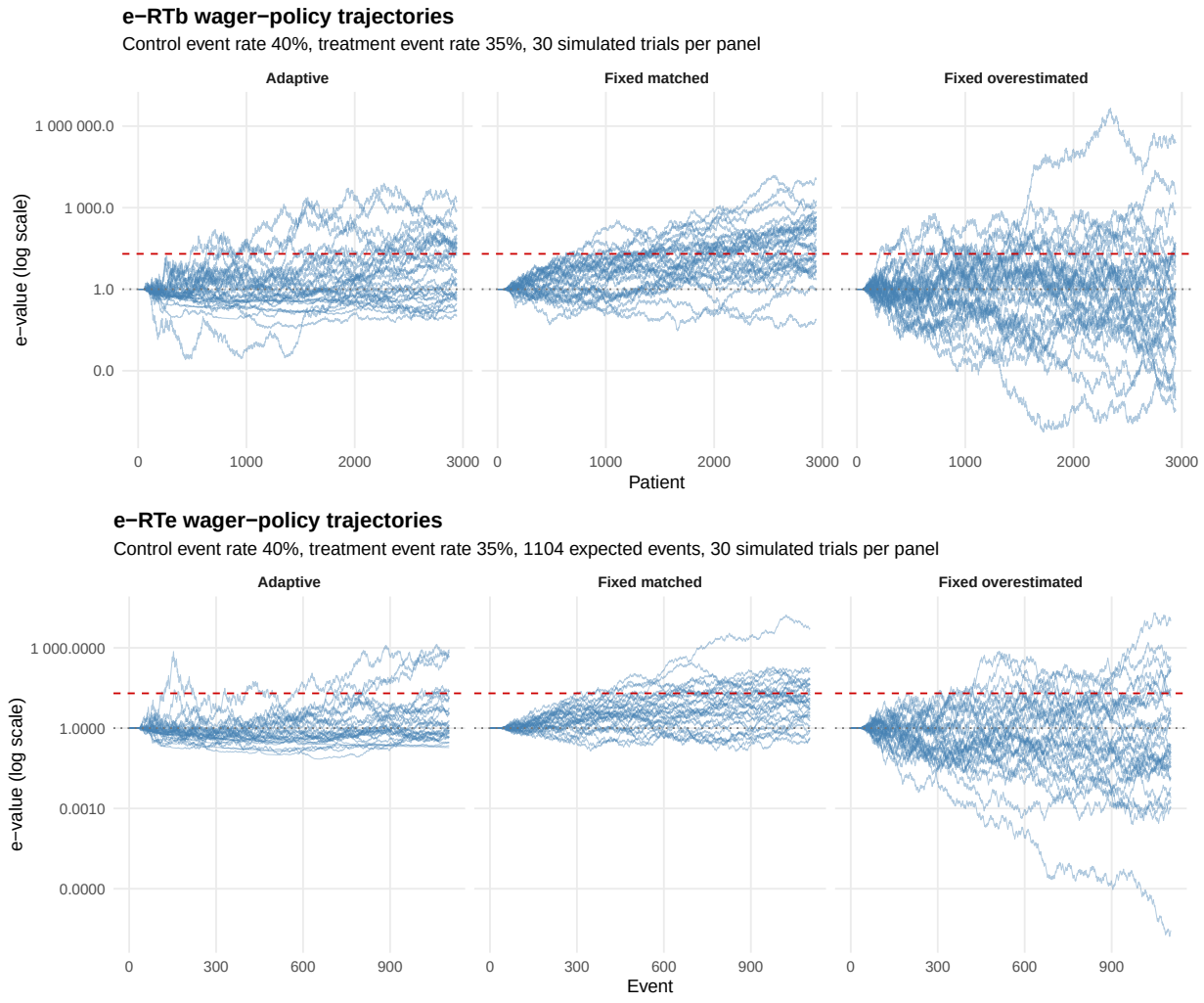

Figure 6: Example wealth trajectories under a true 5pp ARR with control event rate 40%. Top: e-RTb using all enrolled patients. Bottom: e-RTe using only events. Each panel shows 30 simulated trials.

granular.

One may extend this to continuous endpoint which we shall call e-RTC. The validity still comes only from randomization: under the null hypothesis, the distribution of the continuous outcome is the same in both arms, so the outcome does not help to predict treatment assignment.

5.1 Setup

At each enrollment $i = 1, 2, \ldots$, the trial generates two pieces of information. First, the randomization mechanism assigns the patient to one of the two arms, denoted by $T_i \in \{0, 1\}$ (with 0 for control and 1 for intervention), using a known allocation probability p (typically $p = 0.5$ for 1:1 randomization, although equal allocation is not required). Second, once follow-up is complete, we observe a continuous outcome $Y_i \in \mathbb{R}$ such as ventilator–free days, a biomarker concentration, or a physiologic measurement. So far, same as before but using a continuous endpoint.

Under the null hypothesis of no treatment effect, the distribution of Y_i is identical in both arms, and hence the outcome carries no information about the treatment assignment; formally, $Y_i \perp T_i$ under H_0. This single independence relationship is the foundation of the continuous randomization e-process. The idea mirrors the binary approach: each patient creates a small betting game in which we observe Y_i, form a data-driven guess about which arm the patient is more likely to belong to, and then update our wealth once the actual assignment T_i is revealed. If the null is true, these guesses cannot systematically win because outcomes are uninformative about treatment. If the alternative is true, outcomes begin to separate between arms, the bets gain predictive power, and wealth grows accordingly.

Viewed this way, extending the randomization e-process from binary to continuous outcomes may not require additional assumptions; it merely replaces the event/no-event signal with a continuous measure of extremeness relative to past observations, while preserving the anytime-valid martingale structure. The difference in how the wager is defined.

5.2 Betting strategy for continuous outcomes

We can use a data-driven rule with three ingredients:

1. A *center* and *scale* for the outcomes observed so far. Outcomes can be all over the place. We need to standardize.

2. A standardized residual for the new outcome. As the reader will see we bet proportionally to how far the result is away from the center reference we chose. This also needs to be standardized.

3. A smooth map from that residual into a betting fraction $\lambda_i \in (0, 1)$. If the measurement is an outlier, a huge wager would be placed because difference between measurement and the center of the scale would be huge, we need to muffle it to something between $(0, 1)$.

In simpler terms: Each new outcome is first compared to the past outcomes to understand how "unusual" it is. To do this, we anchor the outcome to a robust center and a robust scale: the median gives the center and the MAD (median absolute deviation) gives the spread. We then compute a standardized value: how far above or below the median this new observation is, measured in MAD units. That standardized number is what guides how aggressively we bet. A large positive value means "this looks unusually high compared with past outcomes," a large negative value means the opposite, and values near zero mean "this one looks typical." The point of the MAD is simple: it behaves well even when early data are messy or skewed. It stops a single extreme value from blowing up the bet and keeps the e-process stable while the trial is still young. This likely comes at the cost of reduced power.

At step i, using all previous outcomes $Y_1, \ldots, Y_{i-1}$, we compute:

$$m_{i-1} = \text{median}(Y_1, \ldots, Y_{i-1}), \tag{22}$$

$$s_{i-1} = \text{MAD}(Y_1, \ldots, Y_{i-1}), \tag{23}$$

where MAD is the median absolute deviation. MAD is defined as $\text{MAD} = \text{median}(|Y_i - \text{median}(Y)|)$. These are robust to outliers and skewness. If s_{i-1} is zero or not finite, we set $s_{i-1} = 1$ to avoid degeneracy.

For the new patient, we form a standardized residual:

$$r_i = \frac{Y_i - m_{i-1}}{s_{i-1}}. \tag{24}$$

This means that the patient residual r_i is the observed value (Y_i) minus the median observed so far (s_{i-1} divided by the MAD (s_{i-1}). Note how this uses information for patients before i, keeping the martingale.

We then squash this standardized value into the interval $(-1, 1)$ using

$$g_i = \frac{r_i}{1 + |r_i|}.$$

This is a simple monotone transformation: for moderate values $g_i \approx r_i$, while very large positive or negative residuals are shrunk toward $+1$ or -1. The only purpose of this step is to prevent a single extreme observation from forcing an almost all-in bet.

Next, we ramp up the betting strength over time. Let

$$c_i = \min\left\{1, \max\left(0, \frac{i - \text{burn-in}}{\text{ramp}}\right)\right\}, \tag{25}$$

where `burn-in` is the number of initial patients during which we essentially do not bet, and `ramp` controls how quickly we move from very cautious betting to our maximum aggressiveness. Those concepts are exactly like the binary approach. Finally, we cap the maximum betting strength at $c_{\max} \in (0, 0.5]$ to avoid pathological bets.

Additionally, we need a direction estimate: which arm has better outcomes? Using all previous data, we compute the running Cohen's d:

$$\hat{d}_{i-1} = \frac{\bar{Y}_{\text{trt}} - \bar{Y}_{\text{ctrl}}}{s_{\text{pooled}}} \tag{26}$$

clamped to $[-1, 1]$, where $\bar{Y}_{\text{trt}}$ and $\bar{Y}_{\text{ctrl}}$ are the arm-specific means and s_{pooled} is the pooled standard deviation. This is the "doubly adaptive" structure: g_i captures how informative the current observation is, while $\hat{d}_{i-1}$ captures the direction and magnitude of the treatment effect estimated from all past data.

The betting fraction λ_i is then

$$\lambda_i = 0.5 + c_i \cdot c_{\max} \cdot g_i \cdot \hat{d}_{i-1}. \tag{27}$$

By construction, $\lambda_i \in (0, 1)$ and is predictable: it depends only on past outcomes and the new Y_i, not on T_i. In words: if the current observation Y_i is extreme in the direction that past data associate with the treatment arm, λ_i deviates substantially from 0.5, placing a confident bet on treatment. If Y_i is typical, or if past data show no treatment effect ($\hat{d} \approx 0$), λ_i stays near 0.5 and we barely bet.

Intuitively:

- If Y_i is close to the historical median, $g_i \approx 0$ and $\lambda_i \approx 0.5$: we essentially do not bet, regardless of how strong the estimated treatment effect is.
- If Y_i is extreme (large $|g_i|$) *and* past data show a substantial treatment effect (large $|\hat{d}|$), then λ_i deviates substantially from 0.5: we place a confident bet on one arm. The product $g_i \cdot \hat{d}_{i-1}$ determines both direction and magnitude.
- If Y_i is extreme but $\hat{d} \approx 0$ (no estimated effect yet), λ_i stays near 0.5—an unusual observation is not informative if we do not yet know which direction to bet.
- Early in the trial, c_i is small, so even unusual observations lead to mild bets. As data accumulates, c_i approaches 1 and the bets become more confident.

5.3 Wealth update

Treatment is still randomized with probability p of intervention. After we choose λ_i, the wealth updates exactly as in the binary approach:

$$W_i = W_{i-1} \times \begin{cases} \lambda_i / p & \text{if } T_i = 1 \text{ (intervention)} \\ (1 - \lambda_i)/(1 - p) & \text{if } T_i = 0 \text{ (control)} \end{cases} \tag{28}$$

with $W_0 = 1$.

The only difference from the binary case is how we choose λ_i, as we saw. For binary outcomes, λ_i depends on the event indicator and past event rates by arm. For continuous outcomes, λ_i depends on how extreme Y_i is relative to past outcomes.

5.4 Worked intuition

Imagine a trial where the outcome is ventilator-free days, and higher is better. Suppose that after 100 patients, the median and MAD of Y are roughly stable, and the running Cohen's d estimate is approximately 1.0 (a large effect). Patient 101 has an unusually high number of ventilator-free days compared with this distribution. The standardized residual r_{101} is positive and large, so $g_{101} \approx 0.8$ and, after burn-in, $c_{101} \approx 1$. With $c_{\max} = 0.6$ and $\hat{d} \approx 1.0$, we get $\lambda_{101} \approx 0.5 + 0.6 \times 0.8 \times 1.0 \approx 0.98$: we strongly bet that this patient was in the intervention arm. If they indeed were, wealth increases by roughly a factor of $0.98/0.5 \approx 2$ for this one patient. If not, wealth shrinks by $(1 - 0.98)/0.5 \approx 0.04$.

Under the null, high values like this are just as likely in control as in intervention: we win and lose in balance, and wealth does not grow on average. Under a true benefit, such favorable outliers cluster in the intervention arm, and the bets pay off more often than not.

5.5 Validity

The key point is that validity does not depend on the choice of median, MAD, or the specific transformation g_i. It depends only on the fact that:

1. λ_i is chosen *before* observing T_i and depends only on past data and Y_i;
2. under the null, T_i is independent of Y_i with $\mathbb{P}(T_i = 1) = p$.

Theorem 3. *Under the null hypothesis of no treatment effect, the e-process wealth process (W_i) is a test martingale: for all i,*

$$\mathbb{E}[W_i \mid \mathcal{F}_{i-1}] \leq W_{i-1},$$

where $\mathcal{F}_{i-1}$ is the sigma-field generated by all observations up to step $i-1$.

Proof. Condition on $\mathcal{F}_{i-1}$ and Y_i. The bet λ_i is now fixed. Under the null, T_i is independent of Y_i and

$$\mathbb{P}(T_i = 1 \mid \mathcal{F}_{i-1}, Y_i) = p, \quad \mathbb{P}(T_i = 0 \mid \mathcal{F}_{i-1}, Y_i) = 1 - p.$$

The conditional expectation of the wealth multiplier is:

$$\mathbb{E}\left[\frac{W_i}{W_{i-1}} \,\middle|\, \mathcal{F}_{i-1}, Y_i\right] = p \cdot \frac{\lambda_i}{p} + (1-p) \cdot \frac{1-\lambda_i}{1-p} \tag{29}$$

$$= \lambda_i + (1 - \lambda_i) \tag{30}$$

$$= 1. \tag{31}$$

Thus $\mathbb{E}[W_i \mid \mathcal{F}_{i-1}, Y_i] = W_{i-1}$, and taking expectations over Y_i yields $\mathbb{E}[W_i \mid \mathcal{F}_{i-1}] = W_{i-1}$. This shows that (W_i) is a martingale with unit expectation under the null. □

As in the binary case, Ville's inequality implies that for any stopping time τ,

$$\mathbb{P}(W_\tau \geq 1/\alpha) \leq \alpha,$$

so rejecting the null when $W_\tau \geq 1/\alpha$ controls Type I error at level α, regardless of the stopping rule.

5.6 Simulation overview

We evaluated e-RTC using the same design philosophy as for binary approach. For a given standardized effect size (Cohen's d) and target power, we first computed the fixed-sample size required for a two-sample t-test at $\alpha = 0.05$. We then simulated trials with that sample size, assigning patients 1:1 to intervention or control, with outcomes drawn from normal distributions of equal variance and means differing by d under the alternative.

The test was run sequentially with a burn-in period and ramp (burn-in = 50 patients, ramp = 100 patients, $c_{\max} = 0.6$). Under the null (no mean difference between arms), Type I error was close to or below the nominal level. Under the alternative, the implementation for continuous endpoints rejected the null with moderate power and typically crossed the threshold at an intermediate sample size (Table 7). Again, we believe that this may not be a replacement for the fixed-sample t-test but a conservative, anytime-valid monitoring tool that can trigger early stopping when effects are larger or clearer than anticipated.

Table 7: Operating Characteristics for Continuous Outcomes

Cohen's d	Target Power	n	Type I Error	e-process Power	Median Crossing
0.20	80%	788	0.042	14.1%	66
0.40	80%	200	0.038	33.6%	66
0.60	80%	90	0.037	55.1%	63
0.20	90%	1054	0.043	14.1%	67
0.40	90%	266	0.043	34.8%	66
0.60	90%	120	0.045	58.5%	65

A visual representation is shown in Figure 7.

Figure 7 illustrates 30 simulated trajectories of the process for continuous endpoints under a design targeting a standardized effect size of $d = 0.40$ with 80% power. Under the null hypothesis (left panel), wealth fluctuates around 1 and gradually drifts downward as repeated small bets accumulate against noise. No trajectory crosses the rejection threshold of $1/\alpha = 20$. Under the corresponding alternative (right panel), outcomes begin to separate between arms, the bets become systematically correct, and the wealth grows. Some trajectories cross the rejection threshold before the planned sample size, demonstrating the potential for early stopping.

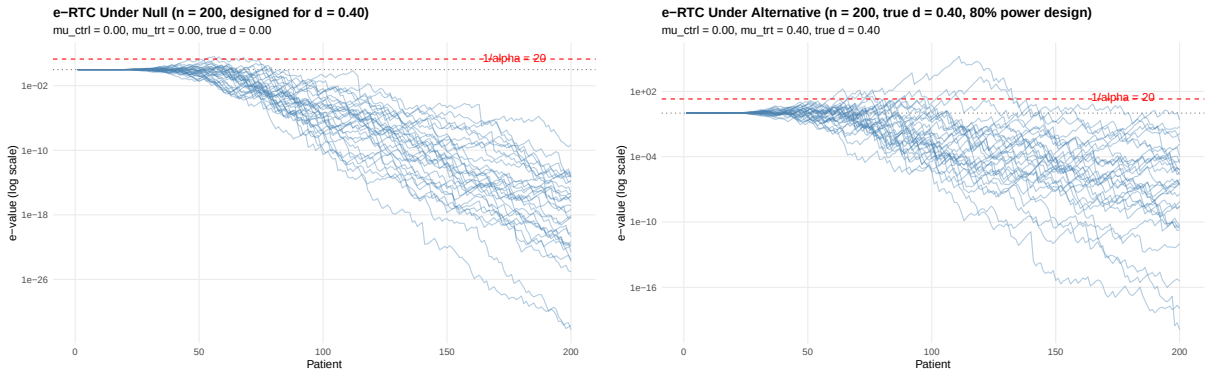

Figure 7: Trajectories of the continuous randomization e-process for a trial designed to detect a standardized mean difference of $d = 0.40$ with 80% power. Left: trajectories under the null hypothesis ($d = 0$), where wealth wanders near or below 1 and rarely approaches the rejection threshold ($1/\alpha = 20$). Right: trajectories under the alternative hypothesis ($d = 0.40$), where wealth grows systematically and many paths cross the rejection threshold before the planned sample size is reached.

6 Time-to-Event Outcomes

Clinical trials often use time-to-event endpoints (e.g., overall survival), usually analyzed via the Log-Rank test or Cox proportional hazards models. These traditional methods require assumptions about proportional hazards or require waiting for a specific number of events. One can extend the randomization e-process to survival data, constructing a sequential Log-Rank test that updates wealth at every observed event.

Grünwald et al. (2021) developed a safe logrank test using e-values under a proportional hazards model with a prior on the hazard ratio. We attempted to construct a nonparametric approach where validity derives solely from randomization, not from a correctly specified hazard model. We call this e-survival.

6.1 Setup and Martingale Construction

Let N patients be randomized to treatment ($T = 1$) or control ($T = 0$). We observe outcomes over time. The "time" scale here is the distinct ordering of events. Let $t_1 < t_2 < \cdots < t_k$ denote the times at which events occur.

At any event time t_j, we define the risk set $\mathcal{R}_j$ as the set of patients who have not yet had an event and have not been censored. Let $Y_1(t_j)$ and $Y_0(t_j)$ be the number of patients at risk in the treatment and control arms, respectively.

Under the null hypothesis of no treatment effect, the probability that the event at time t_j comes from the treatment arm, conditional on a failure occurring within $\mathcal{R}_j$, is simply the proportion of treated patients at risk:

$$p_j = \frac{Y_1(t_j)}{Y_1(t_j) + Y_0(t_j)}. \tag{32}$$

Let X_j be the indicator that the event at t_j is a treated patient ($X_j = 1$ if treated, 0 if control). Under the null, X_j is a Bernoulli trial with probability p_j. We construct the martingale increment (score) as:

$$U_j = X_j - p_j. \tag{33}$$

Note that $\mathbb{E}[U_j|\mathcal{R}_j] = 0$.

6.2 Betting Strategy

We wager on the sign of U_j. If the treatment is beneficial, events will occur more slowly in the treatment arm than expected under the null. Thus, the observed number of treatment events will be lower than the expected number, leading to a negative trend in the cumulative sum of U_j.

We define the cumulative Log-Rank score at step $j-1$ as $Z_{j-1} = \sum_{k=1}^{j-1} U_k$. Our betting strategy targets this trend:

$$\lambda_j = \text{sign}(Z_{j-1}) \cdot c_j \cdot \lambda_{\max}, \tag{34}$$

where $c_j \in [0, 1]$ is a ramping function similar to previous sections, and $\lambda_{\max} < 1$ is a cap on betting aggressiveness.

The wealth update at event j is:

$$W_j = W_{j-1} \times (1 + \lambda_j U_j). \tag{35}$$

Because $\mathbb{E}[U_j] = 0$, the expected multiplicative factor is 1 under the null. Thus, (W_j) is a test martingale.

Note that unlike the binary and continuous approaches, this betting strategy uses only the *sign* of the cumulative score Z_{j-1}, not its magnitude. Once evidence favors one direction, the bet size is fixed at $\lambda_{\max}$ regardless of how strong the accumulated evidence is. This mirrors the Kelly criterion in betting theory: the optimal wager size depends on the expected edge, and $\lambda_{\max} = 0.25$ is calibrated for moderate effects (HR ≈ 0.80) (Kelly, 1956). The connection between the wager, λ, and Kelly's ideas on fraction of betting needs to be further explored. In brief, it makes sense that wager should be higher when prospects of winning are more favorable. For time-to-event, however, adapting response to events may take a long time, and a fixed wager may be preferable.

The parameters burn-in, ramp, and $\lambda_{\max}$ are set arbitrarily (burn-in = 30, ramp = 50, and $\lambda_{\max} = 0.25$ in simulations). Different choices will yield different operating characteristics. The validity of the test does not depend on these choices—only efficiency does.

6.3 Handling Staggered Entry

In clinical practice, patients are recruited over time (staggered entry), whereas this simplified simulation generates survival times simultaneously. However, the Log-Rank test and this betting strategy rely solely on the rank ordering of events based on "time on study."

We verified the validity of this simplification by simulating two scenarios with $N = 631$ and a true

Hazard Ratio of 0.80: (1) simultaneous entry, and (2) staggered entry where patients were recruited uniformly over 12 months, and analysis was performed on calculated study time ($T_{\text{event}} - T_{\text{entry}}$). The resulting distributions of final e-values were similar (Simultaneous Median $E \approx 25.1$, Staggered Median $E \approx 23.6$; Power $\approx 54\%$ and 53% respectively). This confirms that the sequential e-process remains valid for real-world staggered designs provided the analysis utilizes time-since-randomization.

6.4 Simulation Results

We simulated survival trials comparing exponential survival times with a Hazard Ratio (HR) of 0.80. Targeted power was 80% using a standard Log-Rank design, which requires approximately 631 events. A total of $N = 631$ patients (assuming no censoring) were used for simulations. Results are shown in Table 8.

Table 8: Operating Characteristics (N=631)

True HR	Target HR	Type I Error	Power	Median Events to Stop
1.00 (Null)	0.80	0.039	–	–
0.80 (Alt)	0.80	–	62.8%	329 (52% of N)

Under the alternative (HR=0.80), the e-survival process achieved 62.8% power to reject the null, with a median stopping time of 329 events. Examples are shown in Figure 8.

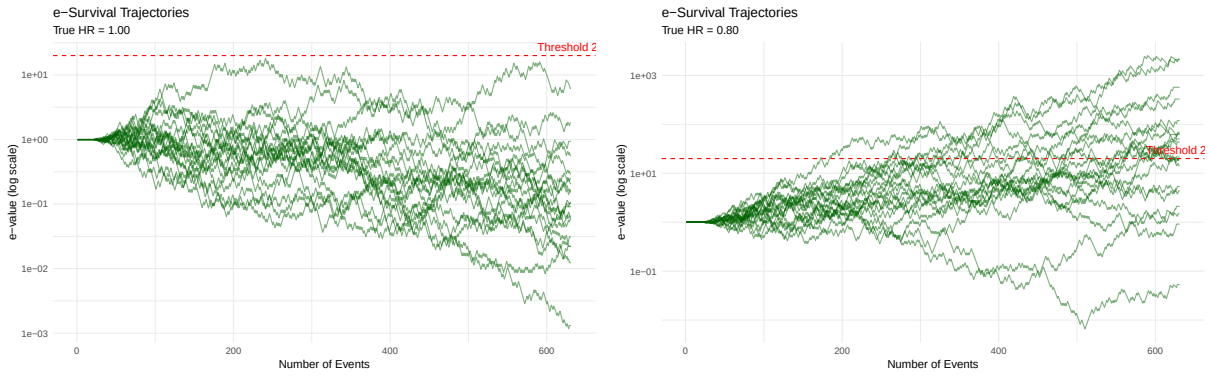

Figure 8: Trajectories of the e-Survival process for a trial designed to detect a Hazard Ratio of 0.80 with 80% power ($N = 631$). Left: trajectories under the null hypothesis (HR = 1.00), where wealth fluctuates randomly. Right: trajectories under the alternative hypothesis (HR = 0.80), where wealth grows systematically. The red dashed line represents the rejection threshold ($1/\alpha = 20$).

7 Multi-State Models

Clinical trials in critical care increasingly use multi-state models to capture patient trajectories. A patient in the ICU may improve to a general ward, be discharged home, or die—and some of these

transitions can reverse. Traditional analyses model the full transition matrix or use competing risks methods, requiring assumptions about the dependence structure between outcomes.

The e-RT framework extends naturally to this setting. Rather than modeling the Markov structure, we classify each transition as "good" (recovery-oriented) or "bad" (deterioration-oriented) and bet on this binary outcome. The method is agnostic to the underlying stochastic process: we simply ask whether good transitions predict treatment assignment.

7.1 Setup

Consider a four-state model common in ICU trials:

- State 1: General Ward
- State 2: ICU
- State 3: Home (absorbing)
- State 4: Dead (absorbing)

Patients begin in the ICU and are followed for a fixed period (e.g., 28 days). Each day, a patient may transition between states according to arm-specific transition probabilities.

We define good transitions as those representing clinical improvement:

- ICU $\to$ Ward (step-down from intensive care)
- Ward $\to$ Home (discharge)

All other transitions—including ICU $\to$ Dead, Ward $\to$ Dead, and Ward $\to$ ICU (readmission)—are classified as bad.

The null hypothesis is that the rate of good transitions is equal between arms:

$$H_0 : P(\text{good transition} \mid \text{treatment}) = P(\text{good transition} \mid \text{control}) \tag{36}$$

7.2 Betting Strategy

Each state transition generates one betting opportunity. Unlike e-RTb where each patient contributes one observation, here a single patient may contribute multiple transitions as they move through states. A patient who goes ICU $\to$ Ward $\to$ ICU $\to$ Ward $\to$ Home contributes four transitions and four bets.

Let $\hat{\delta}_{i-1}$ be the difference in good-transition rates between arms, estimated from all transitions observed before transition i:

$$\hat{\delta}_{i-1} = \frac{\text{good transitions in treatment}}{\text{total transitions in treatment}} - \frac{\text{good transitions in control}}{\text{total transitions in control}} \tag{37}$$

The betting fraction follows the same logic as e-RTb:

$$\lambda_i = \begin{cases} 0.5 + 0.5 \cdot c_i \cdot \hat{\delta}_{i-1} & \text{if transition } i \text{ is good} \\ 0.5 - 0.5 \cdot c_i \cdot \hat{\delta}_{i-1} & \text{if transition } i \text{ is bad} \end{cases} \tag{38}$$

where c_i ramps from 0 to 1 over a burn-in period, exactly as before.

The wealth update remains:

$$W_i = W_{i-1} \times \begin{cases} \lambda_i / 0.5 & \text{if treatment arm} \\ (1 - \lambda_i)/0.5 & \text{if control arm} \end{cases} \tag{39}$$

7.3 Validity

The martingale property holds by the same argument as before. Under the null, treatment assignment is independent of transition quality. Given any transition (good or bad), the probability it came from the treatment arm is 0.5. The expected wealth multiplier is therefore 1, and (W_i) is a test martingale.

The key insight is that validity does not depend on the Markov assumption or any specific transition structure. We have simply replaced "patient with event" with "transition classified as good" and applied the same betting logic. The data-generating process happens to be Markovian in our simulations, but the test would remain valid for any process that generates a sequence of binary-classified transitions from randomized patients.

7.4 Simulation Study

We simulated trials with the following daily transition probabilities:

Table 9: Daily Transition Probabilities

	To State			
From State	Ward	ICU	Home	Dead
Control				
Ward	0.880	0.070	0.030	0.020
ICU	0.070	0.915	0.000	0.015
Treatment				
Ward	0.870	0.050	0.050	0.030
ICU	0.090	0.900	0.000	0.010

Treatment improves recovery transitions: ICU $\rightarrow$ Ward increases from 7% to 9% daily, and Ward $\rightarrow$ Home increases from 3% to 5% daily. The effect on mortality is modest.

All patients started in the ICU and were followed for 28 days. At day 28, the expected state distributions were:

Table 10: Day 28 State Distribution

Arm	Ward	ICU	Home	Dead
Control	20.8%	26.3%	18.8%	34.1%
Treatment	16.9%	16.6%	33.5%	33.0%
Difference	−3.9%	−9.7%	+14.7%	−1.1%

Treatment substantially increases discharge home (+14.7%) while barely changing mortality (−1.1%). This is precisely the scenario where trajectory-based analysis adds value: a mortality-only endpoint would detect little effect, while multi-state analysis captures the clinically meaningful improvement in recovery.

We used burn-in = 30 transitions, ramp = 50 transitions, and threshold $1/\alpha = 20$. Results from 1,000 simulated trials at $N = 1000$ patients:

Table 11: Operating Characteristics for e-RTms ($N = 1000$)

Metric	Value
Type I Error	0.30% (SE: 0.17%)
Power	89.3% (SE: 1.0%)
Median transitions per trial	2017
Median crossing	898 transitions (45%)

Type I error was well below the nominal 5%, consistent with the conservative behavior observed in other e-RT variants. Power was 89% at $N = 1000$; sample size search indicated approximately $N = 800$ for 80% power.

The median trial generated approximately 2000 transitions from 1000 patients (roughly 2 transitions per patient). When the threshold was crossed, it occurred at 898 transitions—45% of the way through the trial.

7.5 Trajectory Examples

Figure 9 shows 30 example wealth trajectories under the null and alternative hypotheses. Under the null (left panel), wealth fluctuates around 1 with the characteristic downward drift from betting on noise. Under the alternative (right panel), wealth grows systematically as good transitions accumulate preferentially in the treatment arm.

7.6 Interpretation

The e-RTms approach offers a philosophically clean handling of multi-state outcomes. By collapsing the transition matrix into a single binary classification, we sidestep the complexities of competing risks and counterfactual reasoning. We do not ask "what would have happened to this patient had they been in the other arm?" We simply ask "does knowing whether this transition was good help

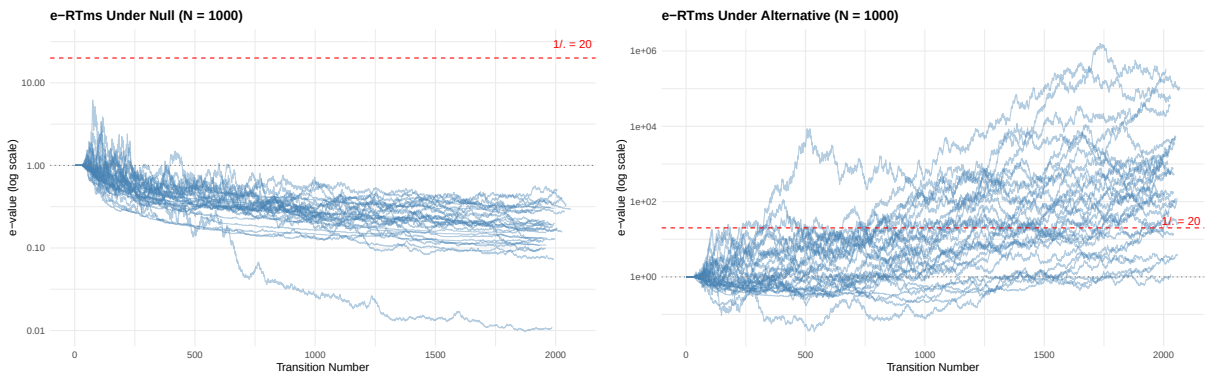

Figure 9: Trajectories of e-RTms for a trial with $N = 1000$ patients. Left: under the null hypothesis (equal transition matrices), wealth fluctuates randomly. Right: under the alternative hypothesis, wealth grows as treatment improves recovery transitions. Dashed red line: rejection threshold ($1/\alpha = 20$).

predict which arm the patient was in?"

This comes at a cost: we lose information about which specific transitions drive the effect. A treatment that dramatically improves ICU → Ward transitions but worsens Ward → Home transitions might show no overall effect if the good-transition rates balance. For monitoring purposes, this limitation is acceptable—the goal is to detect *any* departure from exchangeability, not to dissect the mechanism. Detailed transition-specific analyses can follow at trial completion. e-RT is a monitoring, not a reporting, method.

The method is most valuable when treatment affects trajectory without substantially changing mortality— the pattern in our simulation. For trials where mortality is the dominant signal, e-survival may be more powerful. For trials where the clinical question is "does treatment help patients recover faster?", e-RTms provides a natural framework.

8 Universal e-RT (e-RTu) — Under Development

8.1 Motivation

A recurring pattern across the e-RT variants is that each ultimately reduces to the same core operation: observe a signal, classify it, and bet on which arm it came from. In e-RTb, the signal is an event/no-event indicator. In e-RTe, it is the arm label on an event. In e-RTms, it is a transition classified as good or bad. Even e-RTc, despite its richer continuous signal, squashes the observation through a monotone transformation into a betting fraction that is functionally a soft binary classification.

This observation motivates **e-RTu** (universal): a single, domain-agnostic betting engine that receives a stream of binary-classified signals—each tagged with an arm label and a good/bad indicator—and maintains the wealth process. The upstream logic that generates and classifies signals is separated from the downstream engine that bets on them.

8.2 The Universal Signal

The e-RTu engine sees only a sequence of signals:

$$S_i = (\text{arm}_i, \text{good}_i), \quad \text{arm}_i \in \{\text{treatment}, \text{control}\}, \quad \text{good}_i \in \{\text{true}, \text{false}\} \tag{40}$$

The engine does not know what generated S_i—it could be a patient outcome, a death, a state transition, a biomarker threshold, or any domain-specific event. It maintains running counts of good signals by arm and applies the same half-Kelly adaptive betting strategy as e-RTb:

$$\lambda_i = \begin{cases} 0.5 + 0.5 \cdot c_i \cdot \hat{\delta}_{i-1} & \text{if good}_i = \text{true} \\ 0.5 - 0.5 \cdot c_i \cdot \hat{\delta}_{i-1} & \text{if good}_i = \text{false} \end{cases} \tag{41}$$

where $\hat{\delta}_{i-1}$ is the difference in good-signal rates between arms estimated from all prior signals, and c_i is the standard ramp function.

The wealth update is identical to the binary case:

$$W_i = W_{i-1} \times \begin{cases} \lambda_i / 0.5 & \text{if treatment} \\ (1 - \lambda_i)/0.5 & \text{if control} \end{cases} \tag{42}$$

8.3 Validity

Validity follows from the same argument as all other variants. Under the null hypothesis that arm labels are independent of signal quality, the expected wealth multiplier at each step is exactly 1, making (W_i) a test martingale. The proof is identical to that of e-RTb (Theorem 1). The universality lies in the fact that the proof depends only on the randomization mechanism, not on the domain-specific process that generates the signals.

8.4 Relationship to Other Variants

The e-RTu engine can recover e-RTb and e-RTms exactly:

- **e-RTb**: Each patient generates one signal. Good = event (or non-event, depending on convention). Arm = treatment assignment.
- **e-RTms**: Each state transition generates one signal. Good = recovery-oriented transition. Arm = treatment assignment.

It does *not* recover the specialized strategies of e-RTe (full Kelly), e-RTs (score-based with fixed $\lambda_{\max}$), or e-RTc (doubly adaptive with g-score). These variants exploit domain-specific structure—sparse events, risk sets, continuous residuals—to achieve better power than the generic half-Kelly approach. The wage asymmetry analysis (Section 9) explains why these specializations matter.

The value of e-RTu is conceptual clarity and software simplicity: one engine, many signal sources. It provides a default when no domain-specific variant is available, and a reference implementation against which specialized variants can be benchmarked.

8.5 Status

The e-RTu abstraction is under active development. An R implementation is provided in the supplementary code. Systematic simulation studies comparing e-RTu against specialized variants across domains are planned for a future version.

9 Betting Strategy Design: The Wage Asymmetry

The six e-RT variants use different betting strategies. Binary and continuous methods use adaptive wagers that learn from data; survival uses a fixed wager $\lambda = 0.25$; event-only uses adaptive full Kelly. This section explains why these differences are principled, not accidental.

9.1 The Core Asymmetry: Update Density and Over-Betting

The wealth process is a *product* of multipliers: $W_n = \prod_{i=1}^{n}(1 + b_i u_i)$. When the bet magnitude $|b_i|$ is too large relative to the true effect size ("over-betting"), some multipliers fall well below 1. The critical question is: how fast does wealth recover?

The answer depends on how frequently the product updates:

Survival (e-RTs): Wealth updates only at events (deaths or failures). Between events, wealth is unchanged. A trial with $N = 500$ events produces 500 multiplications. Over-betting makes individual multipliers more volatile, but the limited number of updates means wealth bleeds slowly. In simulation, setting $\lambda = 0.25$ when the true hazard ratio is 0.90 (where the optimal λ is approximately 0.15) yields a median final e-value of approximately 0.04—low, but not zero. The wealth process is damaged but not destroyed.

Binary (e-RTb): Wealth updates at *every* patient. A trial with $N = 2{,}000$ patients produces 2,000 multiplications. Over-betting by 2–3× Kelly and multiplying by values like 0.97 thousands of times compounds to zero irreversibly. In simulation, over-betting at 3× Kelly on a 5 percentage-point ARR gives a median final e-value of exactly 0. The wealth process is destroyed.

Continuous (e-RTc): The problem is even worse. Beyond the dense updates (every patient), the g-score introduces a second source of variability in each wager. Fixed-wage betting is catastrophic at all tested levels for effects of $d \leq 0.20$.

In plain language: imagine a gambler who bets too aggressively. If they play once a week (survival), a bad streak hurts but they can recover. If they play every hour (binary), the same over-bet compounds into ruin. If they play every hour *and* each bet has extra noise on top of the sizing error (continuous), the ruin is faster still.

9.2 Survival: Why Fixed $\lambda = 0.25$ Works

For survival data, the bet has fixed magnitude but adaptive direction:

$$b_i = c_i \times \lambda_{\max} \times \text{sign}(Z_{i-1}) \tag{43}$$

where $\lambda_{\max} = 0.25$ is fixed and Z_{i-1} is the cumulative log-rank score.

Simulation studies across the clinically relevant range of hazard ratios show that $\lambda = 0.25$ is not universally optimal but is robustly adequate:

Table 12: Survival: Fixed $\lambda = 0.25$ vs. Adaptive Half-Kelly

True HR	$\lvert\log(\text{HR})\rvert$	Fixed $\lambda = 0.25$	Adaptive $\frac{1}{2}$-Kelly	Difference
0.70	0.357	43.1%	26.6%	+16.5pp
0.75	0.288	58.9%	35.0%	+23.9pp
0.80	0.223	62.6%	35.9%	+26.7pp
0.85	0.163	55.2%	37.8%	+17.4pp
0.90	0.105	40.5%	41.6%	−1.1pp

Fixed $\lambda = 0.25$ outperforms adaptive half-Kelly by 17–27 percentage points across HR = 0.70–0.85. At HR = 0.90 (a small effect), the strategies are approximately equivalent. The adaptive approach converges correctly to the true log(HR) but produces bets that are systematically too conservative: for HR = 0.80, half-Kelly yields $\lambda \approx 0.11$, which is well below 0.25.

The fixed strategy succeeds because events are sparse. A trial with approximately 500 events tolerates moderate over-betting without catastrophic wealth destruction. Pre-specifying λ is analogous to pre-specifying the alternative hypothesis for sample size calculation—every frequentist trial makes a similar design choice.

9.3 Binary: Why Adaptive Half-Kelly Remains the Default

For binary data, the wager scales with the running risk difference:

$$\lambda_i = 0.5 \pm 0.5 \times c_i \times \hat{\delta}_{i-1} \tag{44}$$

The factor of 0.5 before $c_i\hat{\delta}$ implements half-Kelly betting. Why not use the full estimate, or a fixed wager as the default?

Earlier stress tests demonstrate the catastrophic cost of naively over-betting in binary trials. For a true ARR of 5%, fixed single-magnitude wagers that are too aggressive can destroy wealth:

At twice the appropriate magnitude, the median e-value is already approximately zero—wealth is irreversibly destroyed. Every patient updates the product, and thousands of slightly-wrong multiplications compound into oblivion.

This does not mean that all fixed wagers are invalid or useless. The design-fixed policy in Table 6 uses calibrated assignment probabilities for events and non-events, rather than a single

Table 13: Binary: Cost of Naive Over-Betting (True ARR = 5%, N = 2,942)

Fixed wager magnitude	Relative scale	Power	Median Final E-value
0.05	1×	56.3%	1.15
0.10	2×	21.9%	≈ 0
0.15	3×	16.5%	≈ 0
0.20	4×	10.0%	≈ 0

fixed magnitude. That policy can be much more powerful when the design effect is credible. The caution is practical: no fixed wager is universally efficient, and trialists often power studies for optimistic effects (e.g., ARR = 10%) when the truth may be 3–5%. If the wager inherits that optimism, power can fall despite validity being preserved.

The adaptive half-Kelly wager is therefore the default because it is agnostic to the design alternative. A design-fixed full-Kelly wager is a useful optional mode when the investigator wants the e-process to target a prespecified clinically meaningful effect, but adaptive wagering remains the more robust default when the design effect is uncertain.

9.4 Continuous: The Strongest Case for Adaptive Betting

The continuous e-RTc has a "doubly adaptive" structure:

$$\lambda_i = 0.5 + c_i \times c_{\max} \times g_i \times \hat{d}_{\text{past}} \tag{45}$$

Both g_i (observation-level informativeness) and $\hat{d}_{\text{past}}$ (running Cohen's d) are adaptive. Removing the magnitude of $\hat{d}$ and using only its sign (making λ "fixed" in the direction sense) is catastrophic:

Table 14: Continuous: Adaptive vs. Fixed Direction Magnitude

Cohen's d	N	Adaptive	Fixed $c = 0.3$	Fixed $c = 0.6$
0.50	128	16.8%	15.5%	45.3%
0.30	352	50.1%	50.1%	25.2%
0.20	788	53.9%	29.3%	13.6%

At Cohen's $d = 0.20$ (a typical clinical trial effect size), fixed $c = 0.6$ retains only 13.6% power (vs. 53.9% adaptive), and fixed $c = 0.3$ drops to 29.3%. The median final e-value is ≈ 0 in both cases—wealth is irreversibly destroyed. The g-score adds a second noise source that, combined with dense updates, makes even moderate fixed betting lethal.

9.5 The Design Principle

The hierarchy of over-betting risk is: **continuous** > **binary** ≫ **survival**. This maps directly to how frequently the wealth product is updated. More frequent updates mean more opportunities for over-betting to compound, requiring increasingly conservative or adaptive strategies.

Table 15: Summary: Betting Strategy by e-RT Variant

Property	e-RTs	e-RTe	e-RTb	e-RTc
Update frequency	Events only	Events only	Every patient	Every patient
Over-bet cost	Low (slow bleed)	Moderate	Catastrophic	Catastrophic
Universal λ possible?	Yes	Marginal	No	No
Kelly fraction	Fixed ($\lambda = 0.25$)	Full	Half default	Doubly adaptive

The e-RTe falls between survival and binary. Like survival, it updates only at events, making the product length moderate. This permits full Kelly rather than half-Kelly. However, unlike survival where $\lambda_{\max} = 0.25$ is fixed, e-RTe uses the plug-in $\hat{p}$ directly because the event-coin estimator converges quickly and the natural scale of the problem ($p \in [0, 1]$) bounds the wager without external calibration.

10 Discussion

The e-RT family comprises six nonparametric sequential tests for randomized trials based on the betting framework for e-values (i-bet Duan et al. (2022)). All variants require only that treatment assignment is randomized—no distributional assumptions about outcomes are needed. This makes them robust complements to model-based analyses. The variants cover binary outcomes (e-RTb), event-only monitoring (e-RTe), continuous endpoints (e-RTc), time-to-event analyses (e-RTs), multi-state trajectory data (e-RTms), and a universal abstraction (e-RTu, under development).

10.1 Operating characteristics

Across all five simulation-validated variants, simulations demonstrate proper Type I error control (consistently 2–4% vs. nominal 5%), confirming the theoretical guarantee from the martingale property. Power varies by variant and scenario: for binary outcomes, approximately 50% power for early stopping in trials designed with 80% frequentist power, and 63–66% in trials designed with 90% power. Continuous and survival variants show similar patterns. The multi-state variant achieved 89% power at $N = 1{,}000$ for the specific transition matrix tested. The event-only variant (e-RTe) requires approximately 2.5× sample size inflation compared with the frequentist benchmark, the cost of discarding non-event information.

When early stopping occurs across all variants, it typically happens at approximately 45–56% of the planned sample size or event count.

These results should be interpreted carefully. From a clinical trial perspective, it is uncertain whether these methods can replace traditional frequentist or Bayesian paradigms, but they may provide a continuous monitoring option that requires no alpha spending and no prespecified interim analysis schedule. If the e-process crosses its threshold, one could consider stopping the trial. If it does not cross, the trial may proceed to its planned conclusion and primary analysis.

10.2 Traditional statistics at crossing

When an e-RT crosses its threshold ($\geq 1/\alpha$), one may ask what a traditional frequentist analysis would report at that exact moment. In practice, the standard test will agree: the p-value will be small and the confidence interval will exclude the null. The e-RT is not detecting phantoms.

However, there is a critical distinction. The traditional analysis at crossing is *formally invalid* because the analyst observed the data at multiple time points and stopped at a favorable one. The frequentist p-value is anti-conservative (too small) and the standard confidence interval has less than 95% coverage at this data-driven stopping time. The e-value, by contrast, is valid regardless of the stopping rule—this is the entire point of the anytime-valid guarantee. The e-value itself carries the inferential weight; the traditional statistics at crossing serve only as a sanity check confirming that the e-RT is detecting a real signal, not an artifact.

10.3 What is the null hypothesis being tested?

This approach tests whether treatment assignment can be predicted from outcomes—equivalently, whether outcomes are exchangeable between arms. Under the null, $Y_i \perp T_i$ at each observation: knowing the outcome provides no information about which arm the patient belongs to. This is neither Fisher's sharp null (every individual has exactly zero treatment effect) nor the weak null of equal population means.

A useful analogy is a casino. Under the null, the house is fair: no betting strategy can systematically grow wealth. The e-value quantifies accumulated evidence that the game is beatable. Rejecting the null means we have found a profitable strategy—outcomes predict assignments better than chance. The user is also referenced to the pioneer lessons by Ramdas (2021).

This framing clarifies both the method's strength and its limitation. The strength is generality: any departure from exchangeability—constant effects, heterogeneous effects, or even randomization failures—makes outcomes informative and wealth grows. The limitation is that the test detects only departures that a *cumulative backward-looking* betting strategy can exploit. In practical terms, the null hypothesis is best understood as: "there exists no stable, learnable relationship between outcomes and treatment assignment that persists long enough for an adaptive strategy to exploit." This is appropriate for most trials where effects are expected to be consistent over enrollment, but investigators should be aware that non-stationary effects represent a blind spot.

10.4 Relationship to existing work

The betting framework for hypothesis testing was developed by Shafer (2021). E-values and e-processes have been extensively studied (Vovk and Wang, 2021; Ramdas et al., 2022; Ramdas and Wang, 2025). Duan et al. (2022) introduced interactive rank testing by betting (i-bet), which applies the betting framework directly to randomized experiments: an analyst sequentially bets on treatment assignments based on observed outcomes, with wealth forming a test martingale under the null. The binary approach implements this framework with a specific adaptive betting strategy

tied to outcome values. Betting approaches have been established for estimating means of bounded random variables (Waudby-Smith and Ramdas, 2023). The continuous extension adapts these principles to the two-sample randomization setting using a standardization strategy.

Koning (2025) develops e-values for group invariance, including permutation tests, using batch-based likelihood ratio statistics normalized by permutation expectations. Grünwald et al. (2021) developed the 'Safe Log-rank Test' based on evaluating likelihood ratios with specific priors on the hazard ratio to ensure growth rate optimality. In contrast, the survival approach constructs a linear test martingale directly from the score function using an adaptive betting strategy. Rather than relying on likelihood integration or specific priors, the survival approach process uses a heuristic 'plug-in' estimate of the effect direction, modulated by a ramping function. This offers a computationally simple, algorithmic alternative that derives validity strictly from the randomization probabilities within the risk set, without requiring the full apparatus of partial likelihood theory.

The e-RTb shares the same martingale foundation as i-bet but differs in key respects: it operates prospectively as patients enroll rather than retrospectively on completed data; it requires no covariates or working models; and it uses betting fractions that adapt continuously to running outcome estimates rather than fixed-magnitude wagers guided by covariate-based predictions. This yields a simpler method that may be suitable for real-time trial monitoring.

It is possible that some of the concepts here were discussed by other authors in different contexts that were not immediately available for this author. Reader is encouraged to reach out if that is the case, and the author will happily adjust accordingly.

10.5 Limitations

Several limitations should be noted. This is an experimental method under development. Simulations are not exhaustive, and the operating characteristics reported are specific to the scenarios tested. It is uncertain how these methods would behave in more complex models, including competing risk models.

The methods test only whether there are differences between arms; they do not provide point estimates or confidence intervals. The adaptive learning of $\hat{\delta}$ requires a burn-in period during which little evidence accumulates. For trials where parametric assumptions are plausible, model-based sequential methods will generally have better power. Our simulations used specific betting strategies; other choices may yield different operating characteristics. The betting strategy design section provides guidance on strategy selection, but optimal calibration for specific clinical scenarios remains an open question. Finally, it is unclear how these methods will behave when heterogeneity in treatment effects exists or there are temporal instabilities in effect size. Extensions to relative effect size approaches (e.g., odds ratio) are under development.

10.6 Future Directions

Several extensions merit exploration. First, the current betting strategy uses a cumulative estimate $\hat{\delta}$ that weights all historical observations equally. This makes the method vulnerable to time-varying

effects: if treatment benefit reverses to harm mid-trial, the strategy continues betting on stale information and wealth erodes despite continuous violation of exchangeability. Adaptive weighting schemes—such as exponential decay, rolling windows, or hybrid estimators blending long-term and recent signals—could improve robustness to non-stationary effects. Second, the betting intensity could itself adapt to recent performance: increasing λ during sustained wealth growth (exploiting a confirmed edge) and dampening it following drawdowns (protecting against regime change). This mirrors Kelly criterion extensions that incorporate drawdown constraints. These refinements trade power under stable effects against robustness to drift, and the optimal balance likely depends on the clinical context. This is under development.

10.7 Conclusion

E-processes provide conceptually valid anytime-valid sequential inference for randomized trials using only the guarantee of randomization. Its validity is unconditional on the data-generating process, making it a potentially useful tool for trial monitoring. While it trades power for this robustness, it offers a valuable complement to traditional analysis methods.

11 Disclaimers and Version Control

11.1 Disclaimer

This is an experimental method under development. Application to real patients should only be considered under surveillance from an experienced statistician and remain strongly discouraged at this point by the author. The author is not responsible for consequences of use of this method.

11.2 LLM use statement

Large language models were extensively used in this work. The author had the idea that perhaps the e-value and e-process machinery could be used to bet against randomization which would result in a continuous trial monitoring tool. They uploaded the references in this manuscript to Gemini 3.0 Pro for brainstorming, which quickly resulted in a preliminary version. This was refined, tested, and debugged using Claude 4.5 Opus and ChatGPT 5.1 Pro. Gemini 3.0 Pro aided with coding for survival approach. Claude Opus 4.6 aided with the deaths-only extension and the wage asymmetry analysis in V6, and with renaming, generalization of e-RTd to e-RTe, and the e-RTu universal abstraction in V7.

11.3 Acknowledgments

The author is thankful to Aaditya Ramdas for their thoughtful comments on the first version and for pointing out previous literature to the author.

11.4 Version Control

1. First Version (Dec 04, 2025)
2. Second Version (Dec 07, 2025): Minor text adjustments; removed claim on sharp null.
3. Third Version (Dec 11, 2025): Added continuous and survival endpoints; text adjustments.
4. Fourth Version (Dec 17, 2025): Correction on signal direction on e-RTC. Text adjustments.
5. Fifth Version (Dec 31, 2025): Added multi-state extension (e-RTms).
6. Sixth Version (Feb 15, 2026): Added deaths-only monitoring (e-RTd). Added betting strategy design section explaining wage asymmetry across variants. Reconciled e-RTc parameters (burn-in 50, ramp 100) and direction formula (full Cohen's d clamped to $[-1, 1]$). Added traditional statistics at crossing discussion. Updated abstract and introduction to cover all five variants. Reordered sections.
7. Seventh Version (Mar 08, 2026): Renamed binary e-RT to e-RTb after its introduction as the prototype. Generalized e-RTd (deaths-only) to e-RTe (event-only), broadening applicability beyond mortality. Added e-RTu (universal) section describing a domain-agnostic betting engine abstraction (under development). Updated all cross-references and discussion to reflect six variants.

References

Duan, B., Ramdas, A., and Wasserman, L. (2022). Interactive rank testing by betting. In Schölkopf, B., Uhler, C., and Zhang, K., editors, *Proceedings of the First Conference on Causal Learning and Reasoning*, volume 177 of *Proceedings of Machine Learning Research*, pages 201–235. PMLR.

Grünwald, P., Ly, A., Perez-Ortiz, M., and Schure, J. T. (2021). The safe logrank test: Error control under optional stopping, continuation and prior misspecification. In Greiner, R., Kumar, N., Gerds, T. A., and van der Schaar, M., editors, *Proceedings of AAAI Spring Symposium on Survival Prediction - Algorithms, Challenges, and Applications 2021*, volume 146 of *Proceedings of Machine Learning Research*, pages 107–117. PMLR.

Kelly, J. L. (1956). A new interpretation of information rate. *Bell System Technical Journal*, 35(4):917–926.

Koning, N. W. (2025). Measuring evidence against exchangeability and group invariance with e-values. arXiv preprint arXiv:2310.01153.

Ramdas, A. (2021). Game-theoretic probability and statistics (lecture notes). Accessed: 2025-12-09.

Ramdas, A., Ruf, J., Larsson, M., and Koolen, W. M. (2022). Testing exchangeability: Fork-convexity, supermartingales and e-processes. *International Journal of Approximate Reasoning*, 141:83–109.

Ramdas, A. and Wang, R. (2025). Hypothesis testing with e-values. *Foundations and Trends in Statistics*, 1(1-2):1–390.

Shafer, G. (2021). Testing by betting: A strategy for statistical and scientific communication. *Journal of the Royal Statistical Society: Series A*, 184(2):407–431.

Sokolova, A. and Sokolov, V. (2026). E-values for adaptive clinical trials: Anytime-valid monitoring in practice. arXiv preprint arXiv:2602.06379.

Ville, J. (1939). *Étude critique de la notion de collectif*. PhD thesis, Gauthier-Villars, Paris.

Vovk, V. and Wang, R. (2021). E-values: Calibration, combination and applications. *Annals of Statistics*, 49(3):1736–1754.

Waudby-Smith, I. and Ramdas, A. (2023). Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(1):1–27.

A R Code

```
# e-RT: Sequential Randomization Tests Using e-values
# Supplementary R Code

library(tidyverse)

# --- e-RTb: Binary Outcomes ---

compute_eRT <- function(treatment, outcome, p = 0.5, burn_in = 50, ramp =
   100) {
  n <- length(treatment)
  if (is.factor(treatment)) treatment <- as.numeric(treatment) - 1
  if (is.factor(outcome)) outcome <- as.numeric(outcome) - 1

  wealth <- numeric(n)
  wealth[1] <- 1

  for (i in 2:n) {
    trt_prev <- treatment[1:(i-1)]
    out_prev <- outcome[1:(i-1)]

    rate_trt <- mean(out_prev[trt_prev == 1])
```

```
    rate_ctrl <- mean(out_prev[trt_prev == 0])
    if (is.nan(rate_trt)) rate_trt <- 0.5
    if (is.nan(rate_ctrl)) rate_ctrl <- 0.5

    delta_hat <- rate_trt - rate_ctrl
    c_i <- max(0, min(1, (i - burn_in) / ramp))

    lambda <- if (outcome[i] == 1) {
      0.5 + 0.5 * c_i * delta_hat
    } else {
      0.5 - 0.5 * c_i * delta_hat
    }
    lambda <- max(0.001, min(0.999, lambda))

    multiplier <- if (treatment[i] == 1) lambda / p else (1 - lambda) / (1
        - p)
    wealth[i] <- wealth[i-1] * multiplier
  }
  return(wealth)
}

simulate_trial <- function(n, p_trt = 0.5, rate_trt, rate_ctrl) {
  treatment <- rbinom(n, 1, p_trt)
  outcome <- numeric(n)
  outcome[treatment == 1] <- rbinom(sum(treatment == 1), 1, rate_trt)
  outcome[treatment == 0] <- rbinom(sum(treatment == 0), 1, rate_ctrl)
  data.frame(treatment = treatment, outcome = outcome)
}

simulate_eRT <- function(n_sims = 5000, p_ctrl, p_trt = NULL,
                         hypothesized_ARR = NULL, target_power = 0.80,
                         alpha = 0.05, burn_in = 50, ramp = 100) {

  if (is.null(hypothesized_ARR) && is.null(p_trt)) {
    stop("Must␣specify␣either␣p_trt␣or␣hypothesized_ARR")
  }
  if (is.null(hypothesized_ARR)) hypothesized_ARR <- p_ctrl - p_trt

  ss <- power.prop.test(p1 = p_ctrl, p2 = p_ctrl - hypothesized_ARR,
                        power = target_power, sig.level = alpha)
  n_per_arm <- ceiling(ss$n)
  n_patients <- 2 * n_per_arm

  if (is.null(p_trt)) {
```

```
  p_trt <- p_ctrl
  true_ARR <- 0
} else {
  true_ARR <- p_ctrl - p_trt
}

rejections <- 0
first_crossing <- numeric(n_sims)
final_evalues <- numeric(n_sims)

pb <- txtProgressBar(min = 0, max = n_sims, style = 3)
for (sim in 1:n_sims) {
  trial <- simulate_trial(n_patients, rate_trt = p_trt, rate_ctrl = p_
     ctrl)
  wealth <- compute_eRT(trial$treatment, trial$outcome,
                        burn_in = burn_in, ramp = ramp)
  final_evalues[sim] <- wealth[n_patients]

  crossing <- which(wealth >= 1/alpha)
  if (length(crossing) > 0) {
    rejections <- rejections + 1
    first_crossing[sim] <- crossing[1]
  } else {
    first_crossing[sim] <- NA
  }
  setTxtProgressBar(pb, sim)
}
close(pb)

rejection_rate <- rejections / n_sims
se <- sqrt(rejection_rate * (1 - rejection_rate) / n_sims)

list(
  rejection_rate = rejection_rate,
  se = se,
  n_per_arm = n_per_arm,
  n_patients = n_patients,
  p_ctrl = p_ctrl,
  p_trt = p_trt,
  hypothesized_ARR = hypothesized_ARR,
  true_ARR = true_ARR,
  median_crossing = median(first_crossing, na.rm = TRUE),
  first_crossing = first_crossing,
  final_evalues = final_evalues
```

```
  )
}

plot_trajectories <- function(n_trials = 30, p_ctrl, p_trt = NULL,
                              hypothesized_ARR = NULL, target_power =
                                 0.80,
                              alpha = 0.05, burn_in = 50, ramp = 100,
                              title = NULL) {

  if (is.null(hypothesized_ARR) && is.null(p_trt)) {
    stop("Must␣specify␣either␣p_trt␣or␣hypothesized_ARR")
  }
  if (is.null(hypothesized_ARR)) hypothesized_ARR <- p_ctrl - p_trt
  if (is.null(p_trt)) p_trt <- p_ctrl

  ss <- power.prop.test(p1 = p_ctrl, p2 = p_ctrl - hypothesized_ARR,
                        power = target_power, sig.level = alpha)
  n_patients <- 2 * ceiling(ss$n)
  true_ARR <- p_ctrl - p_trt

  if (is.null(title)) {
    title <- if (true_ARR == 0) {
      sprintf("e-RT␣Under␣Null␣(n␣=␣%d,␣designed␣for␣%.0f%%␣ARR)",
              n_patients, hypothesized_ARR * 100)
    } else {
      sprintf("e-RT␣Under␣Alternative␣(n␣=␣%d,␣%.0f%%␣power)",
              n_patients, target_power * 100)
    }
  }

  trajectories <- lapply(1:n_trials, function(i) {
    trial <- simulate_trial(n_patients, rate_trt = p_trt, rate_ctrl = p_
       ctrl)
    wealth <- compute_eRT(trial$treatment, trial$outcome,
                          burn_in = burn_in, ramp = ramp)
    data.frame(patient = 1:n_patients, wealth = wealth, trial = i)
  })

  df <- bind_rows(trajectories)

  ggplot(df, aes(x = patient, y = wealth, group = trial)) +
    geom_line(alpha = 0.4, color = "steelblue") +
    geom_hline(yintercept = 1/alpha, linetype = "dashed", color = "red") +
    geom_hline(yintercept = 1, linetype = "dotted", color = "gray50") +
```

```
    scale_y_log10() +
    labs(title = title,
         subtitle = sprintf("Control = %.0f%%, Treatment = %.0f%%, True
            ARR = %.0f%%",
                            p_ctrl * 100, p_trt * 100, true_ARR * 100),
         x = "Patient", y = "e-value (log scale)") +
    annotate("text", x = n_patients * 0.95, y = 1/alpha * 1.5,
             label = sprintf("1/alpha = %.0f", 1/alpha), color = "red",
                hjust = 1) +
    theme_minimal() +
    theme(plot.title = element_text(face = "bold"), panel.grid.minor =
       element_blank())
}

# --- e-RTe: Event-Only Monitoring ---

compute_eRTe <- function(arms, burn_in = 30, ramp = 50, threshold = 20) {
  n <- length(arms)
  if (is.factor(arms)) arms <- as.numeric(arms) - 1
  arms <- as.numeric(arms)

  wealth <- numeric(n)
  wealth[1] <- 1
  d_trt <- 0; d_ctrl <- 0
  crossed <- FALSE; crossed_at <- NA

  for (i in 1:n) {
    total <- d_trt + d_ctrl
    p_obs <- if (total > 0) d_trt / total else 0.5

    if (i > burn_in && total > 0) {
      c_i <- min(1, max(0, (i - burn_in) / ramp))
      lambda <- 0.5 + c_i * (p_obs - 0.5)
      lambda <- max(0.001, min(0.999, lambda))
    } else {
      lambda <- 0.5
    }

    mult <- if (arms[i] == 1) lambda / 0.5 else (1 - lambda) / 0.5
    wealth[i] <- if (i == 1) mult else wealth[i - 1] * mult

    if (arms[i] == 1) d_trt <- d_trt + 1 else d_ctrl <- d_ctrl + 1
    if (!crossed && wealth[i] >= threshold) {
      crossed <- TRUE; crossed_at <- i
```

```
    }
  }

  total <- d_trt + d_ctrl
  final_p <- if (total > 0) d_trt / total else 0.5
  final_rr <- if (final_p > 0.001 && final_p < 0.999) {
    final_p / (1 - final_p)
  } else if (final_p <= 0.001) { 0 } else { Inf }

  list(wealth = wealth, crossed = crossed, crossed_at = crossed_at,
       final_p = final_p, final_rr = final_rr, d_trt = d_trt, d_ctrl = d_
          ctrl)
}

simulate_events <- function(n_events, p_trt_given_event) {
  rbinom(n_events, 1, p_trt_given_event)
}

event_coin <- function(p_ctrl, p_trt) {
  p_trt / (p_trt + p_ctrl)
}

expected_events <- function(n_patients, p_ctrl, p_trt) {
  ceiling(n_patients / 2 * (p_ctrl + p_trt))
}

simulate_eRTe <- function(n_sims = 1000, p_ctrl, p_trt,
                          n_patients = NULL, target_power = 0.80,
                          alpha = 0.05, burn_in = 30, ramp = 50) {
  threshold <- 1 / alpha
  if (is.null(n_patients)) {
    ss <- power.prop.test(p1 = p_ctrl, p2 = p_trt,
                          power = target_power, sig.level = alpha)
    n_freq <- 2 * ceiling(ss$n)
    n_patients <- ceiling(n_freq * 2.5)
  }
  n_events_null <- expected_events(n_patients, p_ctrl, p_ctrl)
  n_events_alt  <- expected_events(n_patients, p_ctrl, p_trt)
  p_coin_alt <- event_coin(p_ctrl, p_trt)

  null_rej <- 0; alt_rej <- 0
  alt_crossing <- numeric(n_sims)
  for (sim in 1:n_sims) {
    # Null
```

```
    events <- simulate_events(n_events_null, 0.5)
    res <- compute_eRTe(events, burn_in, ramp, threshold)
    if (res$crossed) null_rej <- null_rej + 1
    # Alternative
    events <- simulate_events(n_events_alt, p_coin_alt)
    res <- compute_eRTe(events, burn_in, ramp, threshold)
    if (res$crossed) alt_rej <- alt_rej + 1
    alt_crossing[sim] <- ifelse(res$crossed, res$crossed_at, NA)
  }

  list(type1 = null_rej / n_sims, power = alt_rej / n_sims,
       median_crossing = median(alt_crossing, na.rm = TRUE),
       n_patients = n_patients, n_events_alt = n_events_alt)
}

plot_trajectories_eRTe <- function(n_trials = 30, n_events,
                                   p_trt_given_event,
                                   burn_in = 30, ramp = 50, alpha = 0.05,
                                   title = NULL) {
  threshold <- 1 / alpha
  if (is.null(title)) {
    title <- if (p_trt_given_event == 0.5) {
      sprintf("e-RTe Under Null (p = 0.50, %d events)", n_events)
    } else {
      sprintf("e-RTe Under Alternative (p = %.3f, %d events)",
              p_trt_given_event, n_events)
    }
  }
  trajectories <- lapply(1:n_trials, function(i) {
    events <- simulate_events(n_events, p_trt_given_event)
    res <- compute_eRTe(events, burn_in, ramp, threshold)
    data.frame(event = 1:n_events, wealth = res$wealth, trial = i)
  })
  df <- bind_rows(trajectories)
  ggplot(df, aes(x = event, y = wealth, group = trial)) +
    geom_line(alpha = 0.4, color = "steelblue") +
    geom_hline(yintercept = threshold, linetype = "dashed", color = "red")
        +
    geom_hline(yintercept = 1, linetype = "dotted", color = "gray50") +
    scale_y_log10() +
    labs(title = title, x = "Event Number", y = "e-value (log scale)") +
    annotate("text", x = n_events * 0.95, y = threshold * 1.5,
             label = sprintf("1/alpha = %.0f", threshold),
             color = "red", hjust = 1) +
```

```
    theme_minimal() +
    theme(plot.title = element_text(face = "bold"),
          panel.grid.minor = element_blank())
}

signal_concentration_table <- function(
    baseline_rates = c(0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40),
    arr = 0.05) {
  data.frame(
    baseline = baseline_rates,
    trt_rate = baseline_rates - arr,
    event_coin = sapply(baseline_rates, function(p) event_coin(p, p - arr)
       ),
    tilt_from_half = sapply(baseline_rates,
      function(p) abs(event_coin(p, p - arr) - 0.5)),
    arr = arr
  )
}

# --- e-RTC: Continuous Outcomes ---

compute_eRTC <- function(treatment, outcome, p = 0.5, burn_in = 50,
                         ramp = 100, c_max = 0.6) {
  n <- length(treatment)
  if (is.factor(treatment)) treatment <- as.numeric(treatment) - 1

  wealth <- numeric(n)
  wealth[1] <- 1

  for (i in 2:n) {
    out_prev <- outcome[1:(i-1)]

    if ((i - 1) < burn_in) {
      wealth[i] <- wealth[i-1]
      next
    }

    med_prev <- median(out_prev, na.rm = TRUE)
    mad_prev <- mad(out_prev, center = med_prev, constant = 1, na.rm =
       TRUE)
    if (!is.finite(mad_prev) || mad_prev <= 0) mad_prev <- 1

    y_i <- outcome[i]
    r_i <- y_i - med_prev
```

```
    s_i <- r_i / mad_prev
    g_i <- s_i / (1 + abs(s_i))

    trt_prev <- treatment[1:(i-1)]
    mean_trt <- mean(out_prev[trt_prev == 1])
    mean_ctrl <- mean(out_prev[trt_prev == 0])

    # Cohen's d (clamped to [-1, 1]) -- not just sign
    if (is.nan(mean_trt) || is.nan(mean_ctrl)) {
      d_hat <- 0
    } else {
      sd_trt <- sd(out_prev[trt_prev == 1])
      sd_ctrl <- sd(out_prev[trt_prev == 0])
      if (is.na(sd_trt) || sd_trt == 0) sd_trt <- 1
      if (is.na(sd_ctrl) || sd_ctrl == 0) sd_ctrl <- 1
      s_pooled <- sqrt((sd_trt^2 + sd_ctrl^2) / 2)
      d_hat <- max(-1, min(1, (mean_trt - mean_ctrl) / s_pooled))
    }

    ramp_frac <- max(0, min(1, (i - burn_in) / ramp))
    c_i <- ramp_frac * c_max

    lambda <- 0.5 + c_i * g_i * d_hat
    lambda <- max(0.001, min(0.999, lambda))

    multiplier <- if (treatment[i] == 1) lambda / p else (1 - lambda) / (1
        - p)
    wealth[i] <- wealth[i-1] * multiplier
  }
  return(wealth)
}

simulate_trial_continuous <- function(n, p_trt = 0.5, mu_ctrl = 0, mu_trt
   = 0, sd = 1) {
  treatment <- rbinom(n, 1, p_trt)
  outcome <- numeric(n)
  outcome[treatment == 0] <- rnorm(sum(treatment == 0), mean = mu_ctrl, sd
      = sd)
  outcome[treatment == 1] <- rnorm(sum(treatment == 1), mean = mu_trt, sd
     = sd)
  data.frame(treatment = treatment, outcome = outcome)
}

simulate_eRTC <- function(n_sims = 5000, mu_ctrl = 0, mu_trt = NULL,
```

```
                          hypothesized_d, target_power = 0.80, alpha =
                             0.05,
                          burn_in = 50, ramp = 100, c_max = 0.6) {

if (missing(hypothesized_d)) stop("Must␣specify␣hypothesized_d")

sd_true <- 1
delta_design <- hypothesized_d * sd_true

ss <- power.t.test(delta = delta_design, sd = sd_true, power = target_
   power,
                   sig.level = alpha, type = "two.sample", alternative =
                       "two.sided")
n_per_arm <- ceiling(ss$n)
n_patients <- 2 * n_per_arm

if (is.null(mu_trt)) {
  mu_trt <- mu_ctrl
  true_d <- 0
} else {
  true_d <- (mu_trt - mu_ctrl) / sd_true
}

rejections <- 0
first_crossing <- numeric(n_sims)
final_evalues <- numeric(n_sims)

pb <- txtProgressBar(min = 0, max = n_sims, style = 3)
for (sim in 1:n_sims) {
  trial <- simulate_trial_continuous(n_patients, mu_ctrl = mu_ctrl,
                                     mu_trt = mu_trt, sd = sd_true)
  wealth <- compute_eRTC(trial$treatment, trial$outcome, p = 0.5,
                         burn_in = burn_in, ramp = ramp, c_max = c_max)
  final_evalues[sim] <- wealth[n_patients]

  crossing <- which(wealth >= 1/alpha)
  if (length(crossing) > 0) {
    rejections <- rejections + 1
    first_crossing[sim] <- crossing[1]
  } else {
    first_crossing[sim] <- NA
  }
  setTxtProgressBar(pb, sim)
}
```

```
  close(pb)

  rejection_rate <- rejections / n_sims
  se <- sqrt(rejection_rate * (1 - rejection_rate) / n_sims)

  list(
    rejection_rate = rejection_rate,
    se = se,
    n_per_arm = n_per_arm,
    n_patients = n_patients,
    mu_ctrl = mu_ctrl,
    mu_trt = mu_trt,
    hypothesized_d = hypothesized_d,
    true_d = true_d,
    median_crossing = median(first_crossing, na.rm = TRUE),
    first_crossing = first_crossing,
    final_evalues = final_evalues
  )
}

plot_trajectories_eRTC <- function(n_trials = 30, mu_ctrl = 0, mu_trt =
   NULL,
                                   hypothesized_d, target_power = 0.80,
                                   alpha = 0.05, burn_in = 50, ramp = 100,
                                   c_max = 0.6, title = NULL) {

  if (missing(hypothesized_d)) stop("Must␣specify␣hypothesized_d")

  sd_true <- 1
  ss <- power.t.test(delta = hypothesized_d, sd = sd_true, power = target_
     power,
                     sig.level = alpha, type = "two.sample", alternative =
                         "two.sided")
  n_patients <- 2 * ceiling(ss$n)

  if (is.null(mu_trt)) {
    mu_trt <- mu_ctrl
    true_d <- 0
  } else {
    true_d <- (mu_trt - mu_ctrl) / sd_true
  }

  if (is.null(title)) {
    title <- if (true_d == 0) {
```

```
      sprintf("e-RTC␣Under␣Null␣(n␣=␣%d,␣designed␣for␣d␣=␣%.2f)", n_
         patients, hypothesized_d)
    } else {
      sprintf("e-RTC␣Under␣Alternative␣(n␣=␣%d,␣true␣d␣=␣%.2f)", n_
         patients, true_d)
    }
  }

  trajectories <- lapply(1:n_trials, function(i) {
    trial <- simulate_trial_continuous(n_patients, mu_ctrl = mu_ctrl,
                                       mu_trt = mu_trt, sd = sd_true)
    wealth <- compute_eRTC(trial$treatment, trial$outcome, p = 0.5,
                           burn_in = burn_in, ramp = ramp, c_max = c_max)
    data.frame(patient = 1:n_patients, wealth = wealth, trial = i)
  })

  df <- bind_rows(trajectories)

  ggplot(df, aes(x = patient, y = wealth, group = trial)) +
    geom_line(alpha = 0.4, color = "steelblue") +
    geom_hline(yintercept = 1/alpha, linetype = "dashed", color = "red") +
    geom_hline(yintercept = 1, linetype = "dotted", color = "gray50") +
    scale_y_log10() +
    labs(title = title,
         subtitle = sprintf("mu_ctrl␣=␣%.2f,␣mu_trt␣=␣%.2f,␣true␣d␣=␣%.2f"
            ,
                            mu_ctrl, mu_trt, true_d),
         x = "Patient", y = "e-value␣(log␣scale)") +
    annotate("text", x = n_patients * 0.95, y = (1/alpha) * 1.5,
             label = sprintf("1/alpha␣=␣%.0f", 1/alpha), color = "red",
                hjust = 1) +
    theme_minimal() +
    theme(plot.title = element_text(face = "bold"), panel.grid.minor =
       element_blank())
}

# --- e-Survival: Time-to-Event Outcomes ---

library(survival)

compute_eSurvival <- function(time, status, treatment,
                              burn_in = 30, ramp = 50, lambda_max = 0.25)
                                 {
  n <- length(time)
```

```
df <- data.frame(time, status, treatment) %>% arrange(time)
T_sorted <- df$time
S_sorted <- df$status
A_sorted <- df$treatment

wealth <- numeric(n)
wealth[1] <- 1
cumulative_Z <- 0

risk_trt <- sum(treatment == 1)
risk_ctrl <- sum(treatment == 0)

for (i in 1:n) {
  if (i > burn_in) {
    c_i <- max(0, min(1, (i - burn_in) / ramp))
    bet_direction <- sign(cumulative_Z)
    b_i <- c_i * lambda_max * bet_direction
  } else {
    b_i <- 0
  }

  is_event <- S_sorted[i] == 1
  tr_is_event <- (A_sorted[i] == 1)

  total_risk <- risk_trt + risk_ctrl
  p_null <- if (total_risk > 0) risk_trt / total_risk else 0.5

  if (is_event) {
    obs <- ifelse(tr_is_event, 1, 0)
    U_i <- obs - p_null
    multiplier <- 1 + b_i * U_i
    cumulative_Z <- cumulative_Z + U_i
    wealth[i] <- if (i > 1) wealth[i-1] * multiplier else multiplier
  } else {
    if (i > 1) wealth[i] <- wealth[i-1]
  }

  if (tr_is_event) {
    risk_trt <- max(0, risk_trt - 1)
  } else {
    risk_ctrl <- max(0, risk_ctrl - 1)
  }
}
```

```
  data.frame(event_num = 1:n, time = T_sorted, wealth = wealth)
}

simulate_trial_survival <- function(n, HR = 1, shape = 1.2, scale = 10,
   cens_prop = 0.0) {
  treatment <- rbinom(n, 1, 0.5)
  scale_trt <- scale / (HR^(1/shape))

  U <- runif(n)
  true_time <- numeric(n)
  true_time[treatment == 0] <- scale * (-log(U[treatment == 0]))^(1/shape)
  true_time[treatment == 1] <- scale_trt * (-log(U[treatment == 1]))^(1/
     shape)

  if (cens_prop > 0) {
    C <- runif(n, 0, 2 * scale)
    time <- pmin(true_time, C)
    status <- as.numeric(true_time <= C)
  } else {
    time <- true_time
    status <- rep(1, n)
  }

  list(time = time, status = status, treatment = treatment)
}

simulate_eSurvival <- function(n_sims = 1000, n_patients = NULL, HR_true =
    1,
                               target_HR = 0.7, target_power = 0.80, alpha
                                   = 0.05,
                               burn_in = 30, ramp = 50, lambda_max = 0.25)
                                   {

  if (is.null(n_patients)) {
    z_alpha <- qnorm(1 - alpha/2)
    z_beta <- qnorm(target_power)
    n_patients <- ceiling(4 * ((z_alpha + z_beta) / log(target_HR))^2)
  }

  rejections <- 0
  first_crossing <- numeric(n_sims)

  pb <- txtProgressBar(min = 0, max = n_sims, style = 3)
```

```
  for (sim in 1:n_sims) {
    trial <- simulate_trial_survival(n_patients, HR = HR_true)
    res <- compute_eSurvival(trial$time, trial$status, trial$treatment,
                             burn_in = burn_in, ramp = ramp, lambda_max =
                                lambda_max)

    crossing <- which(res$wealth >= 1/alpha)
    if (length(crossing) > 0) {
      rejections <- rejections + 1
      first_crossing[sim] <- crossing[1]
    } else {
      first_crossing[sim] <- NA
    }
    setTxtProgressBar(pb, sim)
  }
  close(pb)

  rate <- rejections / n_sims
  se <- sqrt(rate * (1 - rate) / n_sims)

  list(rate = rate, se = se, median_stop = median(first_crossing, na.rm =
     TRUE),
       n_patients = n_patients)
}

plot_trajectories_surv <- function(n_trials = 20, n_patients = 400, HR_
   true = 0.7,
                                   alpha = 0.05, title = "e-Survival␣
                                      Trajectories") {

  trajectories <- lapply(1:n_trials, function(i) {
    trial <- simulate_trial_survival(n_patients, HR = HR_true)
    res <- compute_eSurvival(trial$time, trial$status, trial$treatment)
    res$trial <- i
    res
  })

  df <- bind_rows(trajectories)

  ggplot(df, aes(x = event_num, y = wealth, group = trial)) +
    geom_line(alpha = 0.5, color = "darkgreen") +
    geom_hline(yintercept = 1/alpha, linetype = "dashed", color = "red") +
    scale_y_log10() +
    labs(title = title, subtitle = sprintf("True␣HR␣=␣%.2f", HR_true),
```

```
         x = "Number␣of␣Events", y = "e-value␣(log␣scale)") +
    annotate("text", x = n_patients, y = 1/alpha,
             label = sprintf("Threshold␣%.0f", 1/alpha), vjust = -0.5,
                color = "red") +
    theme_minimal()
}

# --- Staggered vs Simultaneous Entry Verification ---

verify_staggered_equivalence <- function(n_sims = 1000, n_patients = 631,
                                         HR_true = 0.8, recruit_period =
                                            12,
                                         alpha = 0.05) {

  final_e_simultaneous <- numeric(n_sims)
  final_e_staggered <- numeric(n_sims)

  pb <- txtProgressBar(min = 0, max = n_sims, style = 3)
  for (i in 1:n_sims) {
    # Simultaneous
    trial_sim <- simulate_trial_survival(n_patients, HR = HR_true)
    res_sim <- compute_eSurvival(trial_sim$time, trial_sim$status, trial_
       sim$treatment)
    final_e_simultaneous[i] <- tail(res_sim$wealth, 1)

    # Staggered
    trial_stag <- simulate_trial_survival(n_patients, HR = HR_true)
    entry_time <- runif(n_patients, 0, recruit_period)
    calculated_study_time <- entry_time + trial_stag$time - entry_time

    res_stag <- compute_eSurvival(calculated_study_time, trial_stag$status
       ,
                                  trial_stag$treatment)
    final_e_staggered[i] <- tail(res_stag$wealth, 1)

    setTxtProgressBar(pb, i)
  }
  close(pb)

  df_res <- data.frame(
    e_value = c(final_e_simultaneous, final_e_staggered),
    Method = rep(c("Simultaneous", "Staggered"), each = n_sims)
  )
```

```
  ggplot(df_res, aes(x = e_value, fill = Method)) +
    geom_density(alpha = 0.5, color = NA) +
    scale_x_log10() +
    labs(title = "Equivalence of Simultaneous vs. Staggered Entry",
         subtitle = sprintf("N=%d, HR=%.2f", n_patients, HR_true),
         x = "Final e-value", y = "Density") +
    theme_minimal() +
    theme(legend.position = "bottom")
}

# --- e-RTms: Multi-State Models ---

# States: 1=Ward, 2=ICU, 3=Home (absorbing), 4=Dead (absorbing)
P_ctrl <- matrix(c(
  0.880, 0.070, 0.030, 0.020,
  0.070, 0.915, 0.000, 0.015,
  0.000, 0.000, 1.000, 0.000,
  0.000, 0.000, 0.000, 1.000
), nrow = 4, byrow = TRUE)

P_trt <- matrix(c(
  0.870, 0.050, 0.050, 0.030,
  0.090, 0.900, 0.000, 0.010,
  0.000, 0.000, 1.000, 0.000,
  0.000, 0.000, 0.000, 1.000
), nrow = 4, byrow = TRUE)

BURN_IN <- 30
RAMP <- 50
MAX_DAYS <- 28
START_STATE <- 2
THRESHOLD <- 20
GOOD_TRANS <- c("2_1", "1_3")

simulate_patient_ms <- function(P, max_days = MAX_DAYS, start_state =
   START_STATE) {
  state <- start_state
  transitions <- NULL

  for (day in 1:max_days) {
    if (state %in% c(3, 4)) break
    new_state <- sample(1:4, 1, prob = P[state, ])
    if (new_state != state) {
```

```
      transitions <- rbind(transitions, c(from = state, to = new_state,
          day = day))
    }
    state <- new_state
  }

  list(final_state = state, transitions = transitions)
}

simulate_trial_ms <- function(n_patients, P_trt, P_ctrl) {
  all_trans <- NULL
  all_arms <- c()
  final_states <- data.frame(patient = 1:n_patients, arm = NA, final_state
      = NA)

  for (i in 1:n_patients) {
    arm <- sample(0:1, 1)
    P <- if (arm == 1) P_trt else P_ctrl
    pat <- simulate_patient_ms(P)

    final_states$arm[i] <- arm
    final_states$final_state[i] <- pat$final_state

    if (!is.null(pat$transitions)) {
      for (j in 1:nrow(pat$transitions)) {
        all_trans <- rbind(all_trans, pat$transitions[j, ])
        all_arms <- c(all_arms, arm)
      }
    }
  }

  list(transitions = all_trans, arms = all_arms, final_states = final_
     states)
}

compute_eRTms <- function(all_transitions, all_arms,
                          burn_in = BURN_IN, ramp = RAMP,
                          good_trans = GOOD_TRANS) {

  n <- nrow(all_transitions)
  if (is.null(n) || n == 0) return(1)

  wealth <- numeric(n)
  wealth[1] <- 1
```

```
  n_good_trt <- 0; n_total_trt <- 0
  n_good_ctrl <- 0; n_total_ctrl <- 0

  for (i in 1:n) {
    trans <- all_transitions[i, ]
    arm <- all_arms[i]
    trans_key <- paste0(trans["from"], "_", trans["to"])
    is_good <- trans_key %in% good_trans

    if (i > burn_in && n_total_trt > 0 && n_total_ctrl > 0) {
      c_i <- min(1, max(0, (i - burn_in) / ramp))
      rate_trt <- n_good_trt / n_total_trt
      rate_ctrl <- n_good_ctrl / n_total_ctrl
      delta <- rate_trt - rate_ctrl

      lambda <- if (is_good) 0.5 + 0.5 * c_i * delta else 0.5 - 0.5 * c_i
          * delta
    } else {
      lambda <- 0.5
    }

    lambda <- max(0.01, min(0.99, lambda))
    mult <- if (arm == 1) lambda / 0.5 else (1 - lambda) / 0.5
    wealth[i] <- if (i > 1) wealth[i - 1] * mult else mult

    if (arm == 1) {
      n_total_trt <- n_total_trt + 1
      if (is_good) n_good_trt <- n_good_trt + 1
    } else {
      n_total_ctrl <- n_total_ctrl + 1
      if (is_good) n_good_ctrl <- n_good_ctrl + 1
    }
  }

  return(wealth)
}

run_simulation_ms <- function(n_sims, n_patients, P_trt, P_ctrl,
                              threshold = THRESHOLD, verbose = TRUE) {

  rejections <- 0
  first_crossing <- numeric(n_sims)
  n_transitions <- numeric(n_sims)
```

```
  if (verbose) pb <- txtProgressBar(min = 0, max = n_sims, style = 3)

  for (s in 1:n_sims) {
    trial <- simulate_trial_ms(n_patients, P_trt, P_ctrl)

    if (is.null(trial$transitions) || nrow(trial$transitions) < BURN_IN) {
      first_crossing[s] <- NA
      n_transitions[s] <- 0
      next
    }

    wealth <- compute_eRTms(trial$transitions, trial$arms)
    n_transitions[s] <- length(wealth)

    crossing <- which(wealth >= threshold)
    if (length(crossing) > 0) {
      rejections <- rejections + 1
      first_crossing[s] <- crossing[1]
    } else {
      first_crossing[s] <- NA
    }

    if (verbose) setTxtProgressBar(pb, s)
  }

  if (verbose) close(pb)

  rate <- rejections / n_sims
  se <- sqrt(rate * (1 - rate) / n_sims)

  list(
    rate = rate,
    se = se,
    median_crossing = median(first_crossing, na.rm = TRUE),
    median_transitions = median(n_transitions)
  )
}

plot_trajectories_ms <- function(n_trials = 30, n_patients, P_trt, P_ctrl,
                                 threshold = THRESHOLD, title = "") {

  trajectories <- list()
```

```
  for (i in 1:n_trials) {
    trial <- simulate_trial_ms(n_patients, P_trt, P_ctrl)
    if (!is.null(trial$transitions) && nrow(trial$transitions) > BURN_IN)
        {
      wealth <- compute_eRTms(trial$transitions, trial$arms)
      trajectories[[length(trajectories) + 1]] <- data.frame(
        transition = 1:length(wealth), wealth = wealth, trial = i
      )
    }
  }

  df <- bind_rows(trajectories)

  ggplot(df, aes(x = transition, y = wealth, group = trial)) +
    geom_line(alpha = 0.4, color = "steelblue") +
    geom_hline(yintercept = threshold, linetype = "dashed", color = "red")
        +
    geom_hline(yintercept = 1, linetype = "dotted", color = "gray50") +
    scale_y_log10() +
    labs(title = title, x = "Transition␣Number", y = "e-value␣(log␣scale)"
       ) +
    annotate("text", x = Inf, y = threshold * 1.5,
             label = sprintf("1/alpha␣=␣%.0f", threshold),
             color = "red", hjust = 1.1) +
    theme_minimal() +
    theme(plot.title = element_text(face = "bold"))
}
```